ICPC Library

template
- hash.sh 1
- mathfunc.hpp 1
- random.hpp 1
- run.sh 1
- setup.sh 1
- template.hpp 2

algorithm
- cartesian.hpp 2
- fibonacci.hpp 2
- minplusac.hpp 2
- minpluscc.hpp 2
- mo.hpp 2
- monge.hpp 3
- monotone.hpp 3
- rollbackmo.hpp 3
- simplex.hpp 3

datastructure
- 2dbit.hpp 4
- 2dsegtree.hpp 4
- bit.hpp 5
- chtmonotone.hpp 5
- disjointsparsetable.hpp 6
- dualsegtree.hpp 6
- fastset.hpp 6
- hashmap.hpp 6
- lazysegtree.hpp 7
- lazysegtreeBS.hpp 7
- linecontainer.hpp 8
- pb_ds.hpp 8
- rbst.hpp 8
- segbeats.hpp 8
- segtree.hpp 9
- segtreeBS.hpp 9
- unionfind.hpp 10
- wavelet.hpp 10

graph
- bimatching.hpp 10
- blockcut.hpp 11
- cycledetect.hpp 11
- euler.hpp 11
- general.hpp 12
- hld.hpp 12
- hungarian.hpp 12
- linkcut.hpp 13
- lowlink.hpp 14
- maxflow.hpp 14
- mincostflow.hpp 14
- rerooting.hpp 15
- scc.hpp 16
- twosat.hpp 16

math
- arbitrary.hpp 16
- berlekamp.hpp 16
- bitwise.hpp 17
- bostanmori.hpp 17
- comb.hpp 17
- crt.hpp 17
- enumquotient.hpp 17
- extgcd.hpp 17
- floorsum.hpp 17
- fps.hpp 17
- getprimroot.hpp 19
- miller.hpp 19
- modint.hpp 19
- ntt.hpp 19
- pollard.hpp 20

geometry
- argsort.hpp 20
- ccw.hpp 20
- closest.hpp 20
- convexhull.hpp 20
- enclosing.hpp 20
- farthest.hpp 21
- geometry.hpp 21
- halfplane.hpp 22
- iscontain.hpp 23

string
- kmp.hpp 23
- manacher.hpp 23
- palindromictree.hpp 23
- rollinghash.hpp 23
- run.hpp 24
- suffixarray.hpp 24
- zalgo.hpp 24

memo
- kazoeage.md 25
- matching.md 25
- optimization.md 25

template

hash.sh

```
# 使い方: sh hash.sh -> コピペ -> Ctrl + D
# コメント・空白・改行を削除して md5 でハッシュする
g++ -dD -E -P -fpreprocessed - | tr -d '[:space:]' | md5sum
| cut -c-6
```

mathfunc.hpp

md5: a61aa7

```
template <typename T, typename U> T ceil(T x, U y) {
    assert(y != 0);
    if (y < 0)
        x = -x, y = -y;
    return (x > 0 ? (x + y - 1) / y : x / y);
}
template <typename T, typename U> T floor(T x, U y) {
    assert(y != 0);
    if (y < 0)
        x = -x, y = -y;
    return (x > 0 ? x / y : (x - y + 1) / y);
}
template <typename T> int popcnt(T x) {
    return __builtin_popcountll(x);
}
template <typename T> int topbit(T x) {
    return (x == 0 ? -1 : 63 - __builtin_clzll(x));
}
template <typename T> int lowbit(T x) {
    return (x == 0 ? -1 : __builtin_ctzll(x));
}
```

random.hpp

md5: 07197a

```
ll RNG(ll L, ll R) { // [L,R]
    static mt19937_64 gen(

chrono::steady_clock::now().time_since_epoch().count());
    return uniform_int_distribution<ll>(L, R)(gen);
}
```

run.sh

```
#!bin/bash
if ! g++ -std=gnu++2a -Wall -Wextra -Wshadow -DGLIBCXX_DEBUG
-DLOCAL -ftrapv $1.cpp; then
    echo CE
    return
fi

for f in samples-$1/*.in
do
    if ! ./a.out < $f > temp; then
        echo RE on $f
        continue
    fi
    if ! diff ${f/in/ans} temp; then
        echo WA on $f
        continue
    fi
    echo passed $f
done
```

setup.sh

```
#!bin/bash
for i in {A..K}
do
    unzip ~/Donwloads/samples-$i.zip -d $i
    touch $i.cpp
done
```

template.hpp

md5: 69346c

```cpp
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for (int i = (int)(a); i < (int)(b);
i++)
#define rrep(i, a, b) for (int i = (int)(b)-1; i >= (int)
(a); i--)
#define ALL(v) (v).begin(), (v).end()
#define SZ(v) (int)v.size()
#define LB(v, x) int(lower_bound(ALL(v), (x)) - (v).begin())
#define UB(v, x) int(upper_bound(ALL(v), (x)) - (v).begin())

using ll = long long int;
using i128 = __int128_t;
const int inf = 0x3fffffff;
const ll INF = 0x1fffffffffffffff;

template <typename T, typename S> bool chmin(T &a, const S
&b) {
    return a > b ? a = b, 1 : 0;
}
template <typename T, typename S> bool chmax(T &a, const S
&b) {
    return a < b ? a = b, 1 : 0;
}

#ifdef LOCAL
#define show(...) _show(0, #__VA_ARGS__, __VA_ARGS__)
#else
#define show(...) true
#endif
template <typename T> void _show(int i, T name) { cerr <<
'\n'; }
template <typename T1, typename T2, typename... T3>
void _show(int i, const T1 &a, const T2 &b, const T3 &...c)
{
    for (; a[i] != ',' && a[i] != '\0'; i++)
        cerr << a[i];
    cerr << ":" << b << " ";
    _show(i + 1, a, c...);
}

// cin.tie(nullptr);
// ios::sync_with_stdio(false);
```

algorithm

cartesian.hpp

md5: f21531

```cpp
vector<int> Cartesian(const vector<int> &a) {
    const int n = a.size();
    vector<int> res(n, -1);
    int cur;
    rep(i, 1, n) {
        cur = i - 1;
        if (a[cur] > a[i]) {
            int pre = cur;
            while (cur != -1 and a[cur] > a[i])
                pre = cur, cur = res[cur];
            if (cur == -1) {
                res[i] = -1;
                res[pre] = i;
            } else {
                res[i] = cur;
                res[pre] = i;
            }
        } else
            res[i] = cur;
    }
    return res;
}
```

fibonacci.hpp

md5: a4ccd0

```cpp
template <typename T, typename F, bool MINIMIZE = 1>
T FibonacciSearch(ll lb, ll rb, F f) {
    ll s = 1, t = 2;
    while (t < rb - lb + 2) {
        s += t;
        swap(s, t);
    }
    ll L = lb - 1, m1 = L + t - s, m2 = L + s;
    T v1 = f(m1), v2 = f(m2);
    while (m1 != m2) {
        t -= s;
        swap(s, t);
        if (rb < m2 or (MINIMIZE ? v1 >= v2 : v1 <= v2)) {
            m2 = m1;
            v2 = v1;
            m1 = L + t - s;
            v1 = f(m1);
        } else {
            L = m1;
            m1 = m2;
            v1 = v2;
            m2 = L + s;
            v2 = m2 <= rb ? f(m2) : v1;
        }
    }
    return m1;
}
```

minplusac.hpp

md5: 1f82cb

```cpp
template <typename T>
vector<T> MinPlusConvolution_arbitrary_convex(vector<T> &a,
vector<T> &b) {
    int n = a.size(), m = b.size();
    auto cmp = [&](int i, int j, int k) -> bool {
        if (i < k)
            return false;
        if (i - j >= m)
            return true;
        return a[j] + b[i - j] >= a[k] + b[i - k];
    };
    auto arg = MonotoneMinima(n + m - 1, n, cmp);
    vector<ll> ret(n + m - 1);
    rep(i, 0, n + m - 1) ret[i] = a[arg[i]] + b[i - arg[i]];
    return ret;
}
```

minpluscc.hpp

md5: 2429eb

```cpp
template <typename T>
vector<T> MinPlusConvolution_convex_convex(vector<T> &a,
vector<T> &b) {
    int n = SZ(a), m = SZ(b);
    vector<T> c(n + m - 1);
    c[0] = a[0] + b[0];
    for (int k = 0, i = 0; k < n + m - 2; k++) {
        int j = k - i;
        if (j == m - 1 or (i < n - 1 and a[i + 1] + b[j] <
a[i] + b[j + 1])) {
            c[k + 1] = a[++i] + b[j];
        } else {
            c[k + 1] = a[i] + b[++j];
        }
    }
    return c;
}
```

mo.hpp

md5: c0198f

```cpp
struct Mo {
    const int n;
    vector<int> L, R;
    Mo(int _n) : n(_n) {}
    void add(int lb, int rb) {
        L.push_back(lb);
        R.push_back(rb);
    }
    template <typename AL, typename AR, typename DL,
typename DR, typename OUT>
    void run(const AL &addl, const AR &addr, const DL &dell,
const DR &delr,
             const OUT &out) {
        const int q = L.size();
        const int w =
            max<int>(1, 1.0 * n / max<double>(1.0, sqrt(q *
2.0 / 3.0)));
        vector<int> ord(q);
        iota(ALL(ord), 0);
        sort(ALL(ord), [&](int i, int j) {
            int a = L[i] / w, b = L[j] / w;
            if (a != b)
```

```
                return a < b;
            if (a & 1)
                return R[i] < R[j];
            else
                return R[i] > R[j];
        });
        int lb = 0, rb = 0;
        for (auto &i : ord) {
            while (lb > L[i])
                addl(--lb);
            while (rb < R[i])
                addr(rb++);
            while (lb < L[i])
                dell(lb++);
            while (rb > R[i])
                delr(--rb);
            out(i);
        }
    }
};
```

monge.hpp

md5: 9a565a

```
template <typename T, T wmx, typename F> T
MongeShortestPath(int n, F f) {
    vector<T> dp(n, wmx);
    dp[0] = 0;
    vector<int> amin(n);
    auto check = [&](int i, int j) {
        T val = dp[j] + f(j, i);
        if (chmin(dp[i], val))
            amin[i] = j;
    };
    auto rec = [&](auto &rec, int L, int R) {
        if (R - L == 1)
            return;
        int mid = (L + R) >> 1;
        rep(k, amin[L], amin[R] + 1) check(mid, k);
        rec(rec, L, mid);
        rep(k, L + 1, mid + 1) check(R, k);
        rec(rec, mid, R);
    };
    rec(rec, 0, n - 1);
    return dp.back();
}

template <typename T, T wmx, typename F>
T dEdgeMongeShortestPath(int n, int d, F f) {
    auto calc = [&](T lamb) -> T {
        T ret = MongeShortestPath<T, wmx>(
                    n, [&](int i, int j) { return f(i, j) +
lamb; }) -
                lamb * d;
        return ret;
    };
    ll lambda = FibonacciSearch<ll>(-wmx * 3, wmx * 3,
calc);
    return calc(lambda);
}
```

monotone.hpp

md5: ed8003

```
template <typename F> vector<int> MonotoneMinima(int R, int
C, F cmp) {
    // cmp(i,j,k) := compare A[i,j] and A[i,k] (A[i,j] ->
false, A[i,k] -> true)
    vector<int> ret(R);
    auto rec = [&](auto &f, vector<int> &target) -> void {
        int m = target.size();
        if (m == 0)
            return;
        vector<int> even;
        for (int i = 1; i < m; i += 2)
            even.push_back(target[i]);
        f(f, even);
        int cur = 0;
        for (int i = 0; i < m; i += 2) {
            ret[target[i]] = cur;
            int end = C - 1;
            if (i != m - 1)
                end = ret[even[i / 2]];
            while (cur < end) {
                cur++;
                if (cmp(target[i], ret[target[i]], cur))
                    ret[target[i]] = cur;
            }
        }
    };
    vector<int> tmp(R);
    iota(ALL(tmp), 0);
    rec(rec, tmp);
    return ret;
}
```

rollbackmo.hpp

md5: 1a3f2f

```
struct RollbackMo {
    using P = array<int, 2>;
    int n;
    vector<P> qs;
    RollbackMo(int _n) : n(_n) {}
    void add(int lb, int rb) { qs.push_back({lb, rb}); }
    template <typename INIT, typename ADD, typename SNAP,
typename ROLL,
              typename OUT>
    void run(const INIT &init, const ADD &insert, const SNAP
&snapshot,
             const ROLL &rollback, const OUT &out) {
        const int q = qs.size();
        const int w = max(1, int(n / sqrt(q + 1)));
        vector<int> ord(q);
        iota(ALL(ord), 0);
        sort(ALL(ord), [&](int i, int j) {
            return P{qs[i][0] / w, qs[i][1]} < P{qs[j][0] /
w, qs[j][1]};
        });
        init();
        snapshot();
        int last = -1, r = 0;
        for (auto &i : ord)
            if (qs[i][1] - qs[i][0] < w) {
                rep(j, qs[i][0], qs[i][1]) insert(j);
                out(i);
                rollback();
            }
        for (auto &i : ord)
            if (qs[i][1] - qs[i][0] >= w) {
                int b = qs[i][0] / w;
                if (last != b) {
                    init();
                    last = b;
                    r = (b + 1) * w;
                }
                while (r < qs[i][1])
                    insert(r++);
                snapshot();
                for (int j = (b + 1) * w - 1; j >= qs[i][0];
j--)
                    insert(j);
                out(i);
                rollback();
            }
    }
};
```

simplex.hpp

md5: e2e4ba

```
#define EPS 1e-10
class Simplex { // max c * x s.t. A*x <= b, x >= 0
  private:
    using Arr = vector<double>;
    using Mat = vector<vector<double>>;
    int *index;
    double **a;
    int row, column, L;
    void Set(const Mat &A, const Arr &b, const Arr &c) {
        infinity = none = false;
        row = A.size(), column = A[0].size() + 1;
        index = new int[row + column];
        int i, j;
        for (i = 0; i < row + column; i++)
            index[i] = i;
        L = row;
        a = new double *[row + 2];
        for (i = 0; i < row + 2; i++)
            a[i] = new double[column + 1];
        for (i = 0; i < row; i++) {
            for (j = 0; j < column - 1; j++)
                a[i][j] = -A[i][j];
            a[i][column - 1] = 1, a[i][column] = b[i];
```

```
            if (a[L][column] > a[i][column])
                L = i;
        }
        for (j = 0; j < column - 1; j++)
            a[row][j] = c[j];
        a[row + 1][column - 1] = -1;
    }
    void solve() {
        int E, i, j;
        int *ls = new int[column + 2];
        for (E = column - 1;;) {
            if (L < row) {
                swap(index[E], index[L + column]);
                a[L][E] = 1 / a[L][E];
                int prv = column + 1;
                for (j = 0; j < column + 1; j++) {
                    if (j != E) {
                        a[L][j] *= -a[L][E];
                        if (abs(a[L][j]) > EPS)
                            ls[prv] = j, prv = j;
                    }
                }
                ls[prv] = column + 1;
                for (i = 0; i < row + 2; i++) {
                    if (abs(a[i][E]) < EPS || i == L)
                        continue;
                    for (j = ls[column + 1]; j < column + 1;
j = ls[j]) {
                        a[i][j] += a[i][E] * a[L][j];
                    }
                    a[i][E] *= a[L][E];
                }
            }
            E = -1;
            for (j = 0; j < column; j++) {
                if (E < 0 || index[E] > index[j]) {
                    if (a[row + 1][j] > EPS ||
                        (abs(a[row + 1][j]) < EPS && a[row]
[j] > EPS))
                        E = j;
                }
            }
            if (E < 0)
                break;
            L = -1;
            for (i = 0; i < row; i++) {
                if (a[i][E] < -EPS) {
                    if (L < 0)
                        L = i;
                    else if (a[L][column] / a[L][E] - a[i]
[column] / a[i][E] <
                            -EPS)
                        L = i;
                    else if (a[L][column] / a[L][E] - a[i]
[column] / a[i][E] <
                                EPS &&
                            index[L] > index[i])
                        L = i;
                }
            }
            if (L < 0) {
                infinity = true;
                return;
            }
        }
        if (a[row + 1][column] < -EPS) {
            none = true;
            return;
        }
        x.assign(column - 1, 0);
        for (i = 0; i < row; i++) {
            if (index[column + i] < column - 1)
                x[index[column + i]] = a[i][column];
        }
        ans = a[row][column];
    }

  public:
    bool infinity, none;
    double ans;
    Arr x;
    Simplex(const Mat &A, const Arr &b, const Arr &c) {
        Set(A, b, c);
        solve();
    }
};
```

datastructure

2dbit.hpp

md5: 4ff00a

```
template <class T> struct BIT2D {
    int n;
    vector<BIT<T>> d;
    vector<int> xs;
    vector<vector<int>> ys;
    vector<pair<ll, ll>> p;
    BIT2D() {}
    void push(ll x, ll y) { p.push_back({x, y}); }
    void init() {
        rep(i, 0, p.size()) xs.push_back(p[i].first);
        sort(ALL(xs));
        xs.erase(unique(ALL(xs)), xs.end());
        n = xs.size() + 1;
        ys.resize(n);
        rep(j, 0, p.size()) {
            int s = upper_bound(ALL(xs), p[j].first) -
xs.begin();
            for (int i = s; i < n; i += (i & -i))
                ys[i].push_back(p[j].second);
        }
        d.resize(n);
        rep(i, 0, n) {
            sort(ALL(ys[i]));
            ys[i].erase(unique(ALL(ys[i])), ys[i].end());
            d[i] = BIT<T>(ys[i].size() + 2);
        }
    }
    void add(ll x, ll y, T z = 1) {
        int s = upper_bound(ALL(xs), x) - xs.begin();
        for (int i = s; i < n; i += (i & -i)) {
            int idx = upper_bound(ALL(ys[i]), y) -
ys[i].begin();
            d[i].add(idx, z);
        }
    }
    T sum(ll x, ll y) {
        T res = 0;
        int s = upper_bound(ALL(xs), x) - xs.begin();
        for (int i = s; i; i -= (i & -i)) {
            int idx = upper_bound(ALL(ys[i]), y) -
ys[i].begin();
            res += d[i].sum(idx + 1);
        }
        return res;
    }
    T sum(ll x1, ll y1, ll x2, ll y2) {
        return sum(x2, y2) - sum(x1 - 1, y2) - sum(x2, y1 -
1) +
                sum(x1 - 1, y1 - 1);
    }
};
```

2dsegtree.hpp

md5: 505ce0

```
template <typename M, typename N, M (*f)(M, M), M (*g)(M,
N), M (*m1)()>
struct SegmentTree2D {
    int n;
    vector<SegmentTree<M, N, f, g, m1>> st;
    vector<vector<int>> ys;
    SegmentTree2D() {}
    int id(int x, int y) { return lower_bound(ALL(ys[x]), y)
- ys[x].begin(); }
    using P = pair<int, int>;
    vector<int> xs;
    vector<P> ps;
    void push(int x, int y) { ps.push_back({x, y}); }
    void init() {
        for (auto &[x, y] : ps)
            xs.push_back(x);
        sort(ALL(xs));
        xs.erase(unique(ALL(xs)), xs.end());
        n = xs.size();
        ys.resize(n * 2);
        for (auto &[x, y] : ps) {
            int i = lower_bound(ALL(xs), x) - xs.begin();
            for (i += n; i; i >>= 1)
                ys[i].push_back(y);
        }
```

```
        for (int i = 0; i < n * 2; i++) {
            sort(ALL(ys[i]));
            ys[i].erase(unique(ALL(ys[i])), ys[i].end());
            st.push_back(SegmentTree<M, N, f, g, m1>
(ys[i].size()));
        }
    }
    void update(int x, int y, N w) {
        x = (lower_bound(ALL(xs), x) - xs.begin()) + n;
        st[x].update(id(x, y), w);
        M cur = st[x][id(x, y)];
        int pos = x;
        while (x >>= 1) {
            M opp = st[pos ^ 1].query(id(pos ^ 1, y), id(pos
^ 1, y + 1));
            cur = f(cur, opp);
            st[x].set(id(x, y), cur);
            pos >>= 1;
        }
    }
    M query(int x0, int x1, int y0, int y1) {
        x0 = lower_bound(ALL(xs), x0) - xs.begin();
        x1 = lower_bound(ALL(xs), x1) - xs.begin();
        M res = m1();
        for (x0 += n, x1 += n; x0 < x1; x0 >>= 1, x1 >>= 1)
{
            if (x0 & 1)
                res = f(res, st[x0].query(id(x0, y0), id(x0,
y1))), x0++;
            if (x1 & 1)
                x1--, res = f(res, st[x1].query(id(x1, y0),
id(x1, y1)));
        }
        return res;
    }
};
```

bit.hpp

md5: 7e4f57

```
template <typename T> struct BIT {
    int n;
    T all = 0;
    vector<T> val;
    BIT(int _n = 0) : n(_n), val(_n + 10) {}
    void clear() {
        val.assign(n + 10, 0);
        all = T();
    }
    void add(int i, T x) {
        for (i++; i <= n; i += (i & -i))
            val[i] = val[i] + x;
        all += x;
    }
    T sum(int i) {
        T res = 0;
        for (; i; i -= (i & -i))
            res += val[i];
        return res;
    }
    T sum(int L, int R) { // [L,R)
        return sum(R) - sum(L);
    }
    int lower_bound(T x) { // sum[0:i]<x,sum[0:i+1]>=x
        int ret = 0, len = 1;
        while (2 * len <= n)
            len <<= 1;
        for (; len >= 1; len >>= 1) {
            if (ret + len <= n and val[ret + len] < x) {
                ret += len;
                x -= val[ret];
            }
        }
        return ret;
    }
};
```

chtmonotone.hpp

md5: 1cc85e

```
template <typename T, bool isMin> struct
ConvexHullTrickAddMonotone {
#define F first
#define S second
    using P = pair<T, T>;
    deque<P> H;
    ConvexHullTrickAddMonotone() = default;
    bool empty() const { return H.empty(); }
    void clear() { H.clear(); }
    inline int sgn(T x) { return x == 0 ? 0 : (x < 0 ? -1 :
1); }
    using D = long double;
    inline bool check(const P &a, const P &b, const P &c) {
        if (b.S == a.S || c.S == b.S)
            return sgn(b.F - a.F) * sgn(c.S - b.S) >=
                   sgn(c.F - b.F) * sgn(b.S - a.S);
        return D(b.F - a.F) * sgn(c.S - b.S) / D(abs(b.S -
a.S)) >=
               D(c.F - b.F) * sgn(b.S - a.S) / D(abs(c.S -
b.S));
    }
    void add(T a, T b) {
        if (!isMin)
            a *= -1, b *= -1;
        P line(a, b);
        if (empty()) {
            H.emplace_front(line);
            return;
        }
        if (H.front().F <= a) {
            if (H.front().F == a) {
                if (H.front().S <= b)
                    return;
                H.pop_front();
            }
            while (H.size() >= 2 && check(line, H.front(),
H[1]))
                H.pop_front();
            H.emplace_front(line);
        } else {
            assert(a <= H.back().F);
            if (H.back().F == a) {
                if (H.back().S <= b)
                    return;
                H.pop_back();
            }
            while (H.size() >= 2 && check(H[H.size() - 2],
H.back(), line))
                H.pop_back();
            H.emplace_back(line);
        }
    }
    inline T get_y(const P &a, const T &x) { return a.F * x
+ a.S; }
    T query(T x) {
        assert(!empty());
        int l = -1, r = H.size() - 1;
        while (l + 1 < r) {
            int m = (l + r) >> 1;
            if (get_y(H[m], x) >= get_y(H[m + 1], x))
                l = m;
            else
                r = m;
        }
        if (isMin)
            return get_y(H[r], x);
        return -get_y(H[r], x);
    }
    T query_monotone_inc(T x) {
        assert(!empty());
        while (H.size() >= 2 && get_y(H.front(), x) >=
get_y(H[1], x))
            H.pop_front();
        if (isMin)
            return get_y(H.front(), x);
        return -get_y(H.front(), x);
    }
    T query_monotone_dec(T x) {
        assert(!empty());
        while (H.size() >= 2 && get_y(H.back(), x) >=
get_y(H[H.size() - 2], x))
            H.pop_back();
        if (isMin)
            return get_y(H.back(), x);
        return -get_y(H.back(), x);
    }
#undef F
#undef S
};
```

disjointsparsetable.hpp

md5: bababa

```
template <typename T, T (*f)(T, T)> struct
DisjointSparseTable {
    vector<vector<T>> buf;
    vector<int> height;
    DisjointSparseTable(const vector<T> &a) {
        int n = a.size(), LG = 0;
        while ((1 << LG) <= n)
            LG++;
        buf.assign(LG, vector<T>(n));
        height.assign(1 << LG, 0);
        rep(i, 2, 1 << LG) height[i] = height[i >> 1] + 1;
        rep(i, 0, n) buf[0][i] = a[i];
        rep(lg, 1, LG) {
            int add = 1 << lg;
            for (int j = 0; j < n; j += (add << 1)) {
                int pos = min(j + add, n);
                buf[lg][pos - 1] = a[pos - 1];
                for (int k = pos - 2; k >= j; k--)
                    buf[lg][k] = f(a[k], buf[lg][k + 1]);
                if (n <= pos)
                    break;
                buf[lg][pos] = a[pos];
                for (int k = pos + 1; k < min(pos + add, n);
k++)
                    buf[lg][k] = f(buf[lg][k - 1], a[k]);
            }
        }
    }
    T query(int L, int R) {
        if (L >= --R)
            return buf[0][L];
        return f(buf[height[L ^ R]][L], buf[height[L ^ R]]
[R]);
    }
};
```

dualsegtree.hpp

md5: b92f67

```
template <typename M, M (*f)(M, M), M (*m1)()> class
DualSegmentTree {
    int sz, height;
    vector<M> data;
    void down(int k) {
        data[k * 2] = f(data[k * 2], data[k]);
        data[k * 2 + 1] = f(data[k * 2 + 1], data[k]);
        data[k] = m1();
    }

  public:
    DualSegmentTree() {}
    DualSegmentTree(int n) {
        sz = 1, height = 0;
        while (sz < n)
            sz <<= 1, height++;
        data.assign(2 * sz, m1());
    }
    void run(vector<M> &v) {
        for (int i = 0; i < (int)v.size(); i++)
            data[i + sz] = v[i];
    }
    void update(int a, int b, M x) {
        if (a >= b)
            return;
        a += sz, b += sz;
        for (int i = height; i; i--) {
            if (((a >> i) << i) != a)
                down(a >> i);
            if (((b >> i) << i) != b)
                down((b - 1) >> i);
        }
        for (; a < b; a >>= 1, b >>= 1) {
            if (a & 1)
                data[a] = f(data[a], x), a++;
            if (b & 1)
                --b, data[b] = f(data[b], x);
        }
    }
    M query(int k) {
        k += sz;
        for (int i = height; i; i--) {
            if (((k >> i) << i) != k)
                down(k >> i);
        }
        M ret = data[k];
        while (k >>= 1)
            ret = f(ret, data[k]);
        return ret;
    }
};
```

fastset.hpp

md5: 4fd92d

```
struct FastSet {
    static constexpr int B = 64;
    int n, lg;
    vector<vector<ull>> seg;
    FastSet(int m) : n(m) {
        do {
            seg.push_back(vector<ull>((m + B - 1) / B));
            m = (m + B - 1) / B;
        } while (m > 1);
        lg = seg.size();
    }
    bool operator[](int i) const { return (seg[0][i / B] >>
(i % B) & 1); }
    void insert(int i) {
        rep(h, 0, lg) {
            seg[h][i / B] |= 1ull << (i % B);
            i /= B;
        }
    }
    void erase(int i) {
        rep(h, 0, lg) {
            seg[h][i / B] &= ~(1ull << (i % B));
            if (seg[h][i / B])
                break;
            i /= B;
        }
    }
    int next(int i) { // smallest >= i
        rep(h, 0, lg) {
            if (i / B == seg[h].size())
                break;
            ull d = seg[h][i / B] >> (i % B);
            if (!d) {
                i = i / B + 1;
                continue;
            }
            i += lowbit(d);
            for (int g = h - 1; g >= 0; g--) {
                i *= B;
                i += lowbit(seg[g][i / B]);
            }
            return i;
        }
        return -1;
    }
    int prev(int i) { // largest <= i
        rep(h, 0, lg) {
            if (i == -1)
                break;
            ull d = seg[h][i / B] << (63 - i % 64);
            if (!d) {
                i = i / B - 1;
                continue;
            }
            i += topbit(d) - (B - 1);
            for (int g = h - 1; g >= 0; g--) {
                i *= B;
                i += topbit(seg[g][i / B]);
            }
            return i;
        }
        return -1;
    }
};
```

hashmap.hpp

md5: 664539

```
template <typename Key, typename Val, int N = 1 << 20>
struct HashMap {
  private:
    array<Key, N> keys;
    array<Val, N> vals;
    bitset<N> used;
    const int shift;
    static constexpr ull RNG() {
        mt19937_64
```

```
gen(chrono::steady_clock::now().time_since_epoch().count());
        return gen();
    }

  public:
    HashMap() : shift(64 - __lg(N)) {
        static_assert((N & (N - 1)) == 0 && N > 0);
    }
    Val &operator[](const Key &x) {
        static ull r = RNG();
        int hash = (ull(x) * r) >> shift;
        for (;;) {
            if (!used[hash]) {
                keys[hash] = x;
                used[hash] = 1;
                return vals[hash] = Val();
            }
            if (keys[hash] == x)
                return vals[hash];
            hash++;
            hash = (hash + 1) & (N - 1);
        }
    }
};
```

lazysegtree.hpp

md5: 407fff

```
template <typename M, typename N, M (*f)(M, M), M (*g)(M,
N), N (*h)(N, N),
          M (*m1)(), N (*n1)()>
class LazySegmentTree {
    int sz, height;
    vector<M> data;
    vector<N> lazy;
    void update(int k) { data[k] = f(data[k * 2], data[k * 2
+ 1]); }
    void apply(int k, N x) {
        data[k] = g(data[k], x);
        if (k < sz)
            lazy[k] = h(lazy[k], x);
    }
    void down(int k) {
        apply(k * 2, lazy[k]);
        apply(k * 2 + 1, lazy[k]);
        lazy[k] = n1();
    }

  public:
    LazySegmentTree(int n = 0) : LazySegmentTree(vector<M>
(n, m1())) {}
    LazySegmentTree(const vector<M> &a) {
        sz = 1, height = 0;
        while (sz < (int)a.size())
            sz <<= 1, height++;
        data.assign(2 * sz, m1());
        lazy.assign(sz, n1());
        rep(i, 0, a.size()) data[sz + i] = a[i];
        for (int i = sz - 1; i; i--)
            update(i);
    }
    void set(int k, M x) {
        k += sz;
        for (int i = height; i; i--)
            down(k >> i);
        data[k] = x;
        for (int i = 1; i <= height; i++)
            update(k >> i);
    }
    void update(int L, int R, N x) {
        if (L >= R)
            return;
        L += sz, R += sz;
        for (int i = height; i; i--) {
            if (((L >> i) << i) != L)
                down(L >> i);
            if (((R >> i) << i) != R)
                down((R - 1) >> i);
        }
        int lb = L, rb = R;
        while (L < R) {
            if (L & 1)
                apply(L++, x);
            if (R & 1)
                apply(--R, x);
            L >>= 1;
            R >>= 1;
        }
        L = lb, R = rb;
        for (int i = 1; i <= height; i++) {
            if (((L >> i) << i) != L)
                update(L >> i);
            if (((R >> i) << i) != R)
                update((R - 1) >> i);
        }
    }
    M query(int L, int R) {
        if (L >= R)
            return m1();
        L += sz, R += sz;
        for (int i = height; i; i--) {
            if (((L >> i) << i) != L)
                down(L >> i);
            if (((R >> i) << i) != R)
                down((R - 1) >> i);
        }
        M lb = m1(), rb = m1();
        while (L < R) {
            if (L & 1)
                lb = f(lb, data[L++]);
            if (R & 1)
                rb = f(data[--R], rb);
            L >>= 1;
            R >>= 1;
        }
        return f(lb, rb);
    }
};
```

lazysegtreeBS.hpp

md5: 14ff75

```
template <class F> int max_right(int L, F ch) {
    if (L == n)
        return n;
    L += sz;
    rrep(i, 1, height + 1) down(L >> i);
    M sum = m1();
    do {
        while (L % 2 == 0)
            L >>= 1;
        if (!ch(f(sum, data[L]))) {
            while (L < sz) {
                down(L);
                L <<= 1;
                if (ch(f(sum, data[L])))
                    sum = f(sum, data[L++]);
            }
            return L - sz;
        }
        sum = f(sum, data[L++]);
    } while ((L & -L) != L);
    return n;
}
template <class F> int min_left(int R, F ch) {
    if (R == 0)
        return 0;
    R += sz;
    rrep(i, 1, height + 1) down((R - 1) >> i);
    M sum = m1();
    do {
        R--;
        while (R > 1 and (R & 1))
            R >>= 1;
        if (!ch(f(data[R], sum))) {
            while (R < sz) {
                down(R);
                R = (R * 2 + 1);
                if (ch(f(data[R], sum))) {
                    sum = f(data[R--], sum);
                }
            }
            return R + 1 - sz;
        }
        sum = f(data[R], sum);
    } while ((R & -R) != R);
    return 0;
}
```

linecontainer.hpp

md5: 5bccbc

```
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line &o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator x, iterator y) {
        if (y == end())
            return x->p = inf, 0;
        if (x->k == y->k)
            x->p = x->m > y->m ? inf : -inf;
        else
            x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m) { // y=kx+m
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z))
            z = erase(z);
        if (x != begin() && isect(--x, y))
            isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll getmax(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

pb_ds.hpp

md5: 6ac09d

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
// not a multiset
// find_by_order(k) -> itr of k-th(0-based) element
// order_of_key(k) -> index of lower_bound(k)
using ordered_set = tree<int, null_type, less<int>,
rb_tree_tag,
                         tree_order_statistics_node_update>;
```

rbst.hpp

md5: ace59b

```
template <typename T, T (*f)(T, T), T (*e)()> struct RBST {
    struct Node {
        Node *l, *r;
        int cnt;
        T x, sum;
        Node() = default;
        Node(T x) : x(x), sum(x), l(0), r(0) { cnt = 1; }
    };
    RBST(int n) : pool(n) {}
    int cnt(const Node *t) { return t ? t->cnt : 0; }
    T sum(const Node *t) { return t ? t->sum : e(); }
    Node *update(Node *t) {
        t->cnt = cnt(t->l) + cnt(t->r) + 1;
        t->sum = f(f(sum(t->l), t->x), sum(t->r));
        return t;
    }
    vector<Node> pool;
    int ptr = 0;
    inline Node *alloc(const T &v) {
        if (si(pool) == ptr)
            pool.resize(si(pool) * 2);
        return &(pool[ptr++] = Node(v));
    }
    Node *merge(Node *l, Node *r) {
        if (!l or !r)
            return l ? l : r;
        if (RNG(0, cnt(l) + cnt(r) - 1) < cnt(l)) {
            l->r = merge(l->r, r);
            return update(l);
        }
        r->l = merge(l, r->l);
        return update(r);
    }
    pair<Node *, Node *> split(Node *t, int k) {
        if (!t)
            return {t, t};
        if (k <= cnt(t->l)) {
            auto [l, r] = split(t->l, k);
            t->l = r;
            return {l, update(t)};
        }
        auto [l, r] = split(t->r, k - cnt(t->l) - 1);
        t->r = l;
        return {update(t), r};
    }
    void insert(Node *&t, int k, const T &v) {
        auto [l, r] = split(t, k);
        t = merge(merge(l, alloc(v)), r);
    }
};
```

segbeats.hpp

md5: b837a7

```
struct SegtreeBeats {
    ll op(int type, ll x, ll y) { return type ? min(x, y) :
max(x, y); }
    bool cmp(int type, ll x, ll y) { return type ? x < y : x
> y; }
    struct alignas(32) Node {
        ll sum = 0;
        ll a1[2] = {}, a2[2] = {-INF, INF}, ac[2] = {1, 1},
add = 0;
    };
    vector<Node> v;
    ll n, log, e[3] = {-INF, INF, 0};
    SegtreeBeats() {}
    SegtreeBeats(int n) : SegtreeBeats(vector<ll>(n)) {}
    SegtreeBeats(const vector<ll> &a) {
        n = 1, log = 0;
        while (n < SZ(a))
            n <<= 1, log++;
        v.resize(2 * n);
        rep(i, 0, SZ(a)) {
            v[i + n].sum = v[i + n].a1[0] = v[i + n].a1[1] =
a[i];
        }
        rrep(i, 1, n) update(i);
    }
    // 0 : add, 1 : chmin, 2 : chmax, 3 : update
    template <int cmd> void apply(int l, int r, ll x) {
        if (l == r)
            return;
        l += n, r += n;
        rrep(i, 1, log + 1) {
            if (((l >> i) << i) != l)
                push(l >> i);
            if (((r >> i) << i) != r)
                push((r - 1) >> i);
        }
        {
            int l2 = l, r2 = r;
            while (l < r) {
                if (l & 1)
                    _apply<cmd>(l++, x);
                if (r & 1)
                    _apply<cmd>(--r, x);
                l >>= 1;
                r >>= 1;
            }
            l = l2;
            r = r2;
        }
        rep(i, 1, log + 1) {
            if (((l >> i) << i) != l)
                update(l >> i);
            if (((r >> i) << i) != r)
                update((r - 1) >> i);
        }
    }
    // 0 : max, 1 : min, 2 : sum
    template <int cmd> ll fold(int l, int r) {
        if (l == r)
            return e[cmd];
        l += n, r += n;
        rrep(i, 1, log + 1) {
            if (((l >> i) << i) != l)
                push(l >> i);
```

```
            if (((r >> i) << i) != r)
                push((r - 1) >> i);
        }
        ll lx = e[cmd], rx = e[cmd];
        while (l < r) {
            if (l & 1)
                op<cmd>(lx, v[l++]);
            if (r & 1)
                op<cmd>(rx, v[--r]);
            l >>= 1;
            r >>= 1;
        }
        if constexpr (cmd <= 1)
            lx = op(cmd, lx, rx);
        if constexpr (cmd == 2)
            lx += rx;
        return lx;
    }

  private:
    void update(int k) {
        Node &p = v[k];
        Node &l = v[k * 2 + 0];
        Node &r = v[k * 2 + 1];
        p.sum = l.sum + r.sum;
        rep(t, 0, 2) {
            if (l.a1[t] == r.a1[t]) {
                p.a1[t] = l.a1[t];
                p.a2[t] = op(t, l.a2[t], r.a2[t]);
                p.ac[t] = l.ac[t] + r.ac[t];
            } else {
                bool f = cmp(t, l.a1[t], r.a1[t]);
                p.a1[t] = f ? l.a1[t] : r.a1[t];
                p.ac[t] = f ? l.ac[t] : r.ac[t];
                p.a2[t] = op(t, f ? r.a1[t] : l.a1[t], f ?
l.a2[t] : r.a2[t]);
            }
        }
    }
    void push_add(int k, ll x) {
        Node &p = v[k];
        p.sum += x << (log + __builtin_clz(k) - 31);
        rep(t, 0, 2) {
            p.a1[t] += x;
            if (p.a2[t] != e[t])
                p.a2[t] += x;
        }
        p.add += x;
    }
    void push(int cmd, int k, ll x) {
        Node &p = v[k];
        p.sum += (x - p.a1[cmd]) * p.ac[cmd];
        if (p.a1[cmd ^ 1] == p.a1[cmd])
            p.a1[cmd ^ 1] = x;
        if (p.a2[cmd ^ 1] == p.a1[cmd])
            p.a2[cmd ^ 1] = x;
        p.a1[cmd] = x;
    }
    void push(int k) {
        Node &p = v[k];
        if (p.add) {
            rep(t, 0, 2) push_add(k * 2 + t, p.add);
            p.add = 0;
        }
        rep(t, 0, 2) rep(s, 0, 2) if (cmp(t, v[k * 2 +
s].a1[t], p.a1[t]))
            push(t, k * 2 + s, p.a1[t]);
    }
    void subtree_ch(int cmd, int k, ll x) {
        if (!cmp(cmd, v[k].a1[cmd], x))
            return;
        if (cmp(cmd, x, v[k].a2[cmd])) {
            return push(cmd, k, x);
        }
        push(k);
        rep(t, 0, 2) subtree_ch(cmd, k * 2 + t, x);
        update(k);
    }
    template <int cmd> inline void _apply(int k, ll x) {
        rep(i, 0, 2) if (cmd >> i & 1) subtree_ch(i, k, x);
        if constexpr (cmd == 0)
            push_add(k, x);
    }
    template <int cmd> inline void op(ll &a, const Node &b)
{
        if constexpr (cmd <= 1)
            a = op(cmd, a, b.a1[cmd]);
        if constexpr (cmd == 2)
            a += b.sum;
    }
};
```

segtree.hpp

md5: 92f435

```
template <typename M, typename N, M (*f)(M, M), M (*g)(M,
N), M (*m1)()>
struct SegmentTree {
    int n;
    vector<M> data;
    SegmentTree(int _n = 0) {
        n = 1;
        while (n < _n)
            n <<= 1;
        data.assign(2 * n, m1());
    }
    void run(vector<M> &v) {
        for (int i = 0; i < (int)v.size(); i++)
            data[i + n] = v[i];
        for (int k = n - 1; k > 0; k--)
            data[k] = f(data[2 * k], data[2 * k + 1]);
    }
    void set(int k, const M &x) {
        k += n;
        data[k] = x;
        while (k >>= 1)
            data[k] = f(data[2 * k], data[2 * k + 1]);
    }
    void update(int k, const N &x) {
        k += n;
        data[k] = g(data[k], x);
        while (k >>= 1)
            data[k] = f(data[2 * k], data[2 * k + 1]);
    }
    M query(int a, int b) {
        M L = m1(), R = m1();
        for (a += n, b += n; a < b; a >>= 1, b >>= 1) {
            if (a & 1)
                L = f(L, data[a++]);
            if (b & 1)
                R = f(data[--b], R);
        }
        return f(L, R);
    }
};
```

segtreeBS.hpp

md5: 41e532

```
template <class F> int max_right(int L, F ch) const {
    int l = sz + L, w = 1;
    M ansL = m1();
    for (; L + w <= sz; l >>= 1, w <<= 1)
        if (l & 1) {
            if (not ch(f(ansL, data[l])))
                break;
            ansL = f(ansL, data[l++]);
            L += w;
        }
    while (l <<= 1, w >>= 1) {
        if (L + w <= sz && ch(f(ansL, data[l]))) {
            ansL = f(ansL, data[l++]);
            L += w;
        }
    }
    return L;
}
template <class F> int min_left(int R, F ch) const {
    int r = sz + R, w = 1;
    M ansR = m1();
    for (; R - w >= 0; r >>= 1, w <<= 1)
        if (r & 1) {
            if (not ch(f(data[r - 1], ansR)))
                break;
            ansR = f(data[--r], ansR);
            R -= w;
        }
    while (r <<= 1, w >>= 1) {
        if (R - w >= 0 && ch(f(data[r - 1], ansR))) {
            ansR = f(data[--r], ansR);
            R -= w;
        }
    }
```

```
    return R;
}
```

unionfind.hpp

md5: 592ea1

```
struct UnionFind {
    vector<int> par;
    int n;
    UnionFind() {}
    UnionFind(int _n) : par(_n, -1), n(_n) {}
    int root(int x) { return par[x] < 0 ? x : par[x] =
root(par[x]); }
    bool same(int x, int y) { return root(x) == root(y); }
    int size(int x) { return -par[root(x)]; }
    bool unite(int x, int y) {
        x = root(x), y = root(y);
        if (x == y)
            return false;
        if (size(x) > size(y))
            swap(x, y);
        par[y] += par[x];
        par[x] = y;
        return true;
    }
};
```

wavelet.hpp

md5: 3eee50

```
template <typename T> struct WaveletMatrix {
    struct BitVector {
        vector<unsigned long long> buf;
        vector<int> rui;
        BitVector(const vector<char> &a = {}) {
            int n = a.size();
            buf.assign((n + 63) >> 6, 0);
            rui.assign(buf.size() + 1, 0);
            rep(i, 0, n) if (a[i]) {
                buf[i >> 6] |= 1ull << (i & 63);
                rui[(i >> 6) + 1]++;
            }
            rep(i, 0, buf.size()) rui[i + 1] += rui[i];
        }
        int rank(int k, bool f = 1) {
            int ret = rui[k >> 6] + __builtin_popcountll(
                                        buf[k >> 6] & ((1ull
<< (k & 63)) - 1));
            if (!f)
                return k - ret;
            else
                return ret;
        }
    };
    int N, lg;
    vector<int> mid;
    vector<BitVector> buf;
    WaveletMatrix() {}
    WaveletMatrix(vector<T> a) : N(a.size()), lg(0) {
        T mx = 0;
        for (auto &x : a)
            chmax(mx, x);
        while ((T(1) << lg) <= mx)
            lg++;
        mid.resize(lg);
        buf.resize(lg);
        for (int d = lg - 1; d >= 0; d--) {
            vector<char> add;
            vector nxt(2, vector<T>());
            for (auto &x : a) {
                add.push_back(x >> d & 1);
                nxt[x >> d & 1].push_back(x);
            }
            mid[d] = (int)nxt[0].size();
            buf[d] = BitVector(add);
            swap(a, nxt[0]);
            a.insert(a.end(), ALL(nxt[1]));
        }
    }
    int rank(int L, int R, T x) { // x in [L,R)
        if ((T(1) << lg) <= x)
            return 0;
        for (int d = lg - 1; d >= 0; d--) {
            bool f = (x >> d & 1);
            L = buf[d].rank(L, f) + (f ? mid[d] : 0);
            R = buf[d].rank(R, f) + (f ? mid[d] : 0);
        }
        return R - L;
    }
    T quantile(int L, int R, int k) { // kth elem in [L,R)
        T ret = 0;
        for (int d = lg - 1; d >= 0; d--) {
            int l0 = buf[d].rank(L, 0), r0 = buf[d].rank(R,
0);
            if (k < r0 - l0)
                L = l0, R = r0;
            else {
                k -= r0 - l0;
                ret |= T(1) << d;
                L += mid[d] - l0, R += mid[d] - r0;
            }
        }
        return ret;
    }
    int freq(int L, int R, T x) { // <x in [L,R)
        if ((T(1) << lg) <= x)
            return R - L;
        int ret = 0;
        for (int d = lg - 1; d >= 0; d--) {
            bool f = (x >> d & 1);
            if (f)
                ret += buf[d].rank(R, 0) - buf[d].rank(L,
0);
            L = buf[d].rank(L, f) + (f ? mid[d] : 0);
            R = buf[d].rank(R, f) + (f ? mid[d] : 0);
        }
        return ret;
    }
    // [a,b) in [L,R)
    int freq(int L, int R, T a, T b) { return freq(L, R, b)
- freq(L, R, a); }
    // largest <=x in [L,R)
    T prev(int L, int R, T x) {
        int cnt = freq(L, R, x);
        return cnt == R - L ? T(-1) : quantile(L, R, cnt);
    }
    // smallest >x in [L,R)
    T next(int L, int R, T x) {
        int cnt = freq(L, R, x);
        return cnt == 0 ? T(-1) : quantile(L, R, cnt - 1);
    }
};
```

graph

bimatching.hpp

md5: 5df9b6

```
vector<int> BiMatching(int n, int m, vector<vector<int>> &g)
{
    vector<int> L(n, -1), R(m, -1), d(n);
    auto dfs = [&](auto &dfs, int v) -> bool {
        int nd = exchange(d[v], 0) + 1;
        for (auto &u : g[v]) {
            if (R[u] == -1 or (d[R[u]] == nd and dfs(dfs,
R[u]))) {
                L[v] = u, R[u] = v;
                return 1;
            }
        }
        return 0;
    };
    for (;;) {
        d.assign(n, 0);
        queue<int> que;
        bool ch = 0;
        rep(i, 0, n) if (L[i] == -1) {
            que.push(i);
            d[i] = 1;
        }
        while (!que.empty()) {
            int v = que.front();
            que.pop();
            for (auto &u : g[v]) {
                if (R[u] == -1)
                    ch = 1;
                else if (!d[R[u]]) {
                    d[R[u]] = d[v] + 1;
                    que.push(R[u]);
                }
            }
        }
```

```
        if (!ch)
            break;
        rep(i, 0, n) if (L[i] == -1) dfs(dfs, i);
    }
    return L;
}
```

blockcut.hpp

md5: 50b4c9

```
struct BlockCut {
    using P = pair<int, int>;
    int n;
    vector<vector<int>> g;
    vector<int> low, ord, st, used;
    int nxt, k;
    vector<P> es;
    BlockCut() {}
    BlockCut(int n) : n(n), g(n), low(n), ord(n), used(n),
nxt(n), k(0) {}
    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<vector<int>> run() {
        rep(v, 0, n) if (!used[v]) {
            dfs(v, -1);
            for (auto &x : st)
                es.push_back({nxt, x});
            nxt++;
            st.clear();
        }
        vector ret(nxt, vector<int>());
        for (auto &[u, v] : es) {
            ret[u].push_back(v);
            ret[v].push_back(u);
        }
        return ret;
    }
    void dfs(int v, int p) {
        st.push_back(v);
        used[v] = 1;
        low[v] = ord[v] = k++;
        int ch = 0;
        for (auto &to : g[v])
            if (to != p) {
                if (!used[to]) {
                    ch++;
                    int s = SZ(st);
                    dfs(to, v);
                    chmin(low[v], low[to]);
                    if ((p == -1 and ch > 1) or
                        (p != -1 and low[to] >= ord[v])) {
                        es.push_back({nxt, v});
                        while (SZ(st) > s) {
                            es.push_back({nxt, st.back()});
                            st.pop_back();
                        }
                        nxt++;
                    }
                } else {
                    chmin(low[v], ord[to]);
                }
            }
    }
};
```

cycledetect.hpp

md5: 82c253

```
template <bool directed> struct CycleDetect {
    using P = pair<int, int>;
    int n, m;
    vector<vector<P>> g;
    vector<P> cycle;
    CycleDetect() {}
    CycleDetect(int _n) : n(_n), m(0), g(n), used(n) {}
    void add_edge(int u, int v) {
        g[u].push_back({v, m});
        if (!directed)
            g[v].push_back({u, m});
        m++;
    }
    void run() {
        rep(i, 0, n) if (dfs(i, -1) == -2) {
            reverse(ALL(cycle));
            break;
        }
    }

  private:
    vector<int> used;          // 0:not,1:seen,2:visited
    int dfs(int v, int pid) { // -1:over,-2:done
        if (used[v] == 1)
            return v;
        if (used[v] == 2)
            return -1;
        used[v] = 1;
        for (auto &[to, id] : g[v])
            if (id != pid) {
                int nxt = dfs(to, id);
                if (nxt != -1) {
                    if (nxt == -2)
                        return -2;
                    cycle.push_back({to, id});
                    if (nxt == v)
                        return -2;
                    return nxt;
                }
            }
        used[v] = 2;
        return -1;
    }
};
```

euler.hpp

md5: 698f86

```
template <bool directed> struct EulerianTrail {
    using P = pair<int, int>;
    int n, m;
    vector<vector<P>> g;
    UnionFind uni;
    vector<P> es;
    vector<int> used, deg;
    EulerianTrail() {}
    EulerianTrail(int N) : n(N), m(0), g(n), uni(n), deg(n)
{}
    void add_edge(int u, int v) {
        es.push_back({u, v});
        g[u].push_back({v, m});
        uni.unite(u, v);
        if (directed) {
            deg[u]++;
            deg[v]--;
        } else {
            g[v].push_back({u, m});
            deg[u]++;
            deg[v]++;
        }
        m++;
    }
    pair<vector<int>, vector<vector<int>>> run() {
        map<int, vector<int>> grps;
        rep(v, 0, n) grps[uni.root(v)].push_back(v);
        vector<int> start;
        vector<vector<int>> ret;
        used.assign(m, 0);
        for (auto &[_, vs] : grps) {
            int S = -1;
            if (directed) {
                for (auto &v : vs) {
                    if (abs(deg[v]) > 1)
                        return {};
                    else if (deg[v] == 1) {
                        if (S == -1)
                            S = v;
                        else
                            return {};
                    }
                }
            } else {
                int odd = 0;
                for (auto &v : vs) {
                    if (deg[v] & 1) {
                        S = v;
                        odd++;
                    }
                }
                if (odd > 2)
                    return {};
            }
```

```
            if (S == -1)
                S = vs.front();
            auto add = get(S);
            if (add.size()) {
                start.push_back(S);
                ret.push_back(add);
            }
        }
        return {start, ret};
    }

  private:
    vector<int> get(int s) {
        stack<P> st;
        vector<int> ret;
        st.push({s, -1});
        while (!st.empty()) {
            int v = st.top().first;
            if (g[v].empty()) {
                ret.push_back(st.top().second);
                st.pop();
            } else {
                auto &e = g[v].back();
                g[v].pop_back();
                if (used[e.second])
                    continue;
                used[e.second] = 1;
                st.push(e);
            }
        }
        ret.pop_back();
        reverse(ALL(ret));
        return ret;
    }
};
```

general.hpp

md5: 64bf8b

```
vector<pair<int, int>> GeneralMatch(int n, vector<pair<int,
int>> &es) {
    mt19937 rnd;
    vector<vector<int>> g(n + 1);
    for (auto &[u, v] : es) {
        g[u + 1].push_back(v + 1);
        g[v + 1].push_back(u + 1);
    }
    vector<int> used(n + 1), mate(n + 1);
    int T;
    auto dfs = [&](auto &dfs, int v) -> bool {
        used[v] = T;
        shuffle(ALL(g[v]), rnd);
        for (auto &u : g[v]) {
            int w = mate[u];
            if (used[w] < T) {
                mate[v] = u, mate[u] = v, mate[w] = 0;
                if (!w or dfs(dfs, w))
                    return 1;
                mate[u] = w, mate[w] = u, mate[v] = 0;
            }
        }
        return 0;
    };
    rep(_, 0, 10) {
        used.assign(n + 1, 0);
        rep(v, 1, n + 1) if (!mate[v]) {
            T++;
            dfs(dfs, v);
        }
    }
    vector<pair<int, int>> ret;
    rep(v, 1, n + 1) if (v < mate[v]) ret.push_back({v - 1,
mate[v] - 1});
    return ret;
}
```

hld.hpp

md5: cee6a1

```
struct HLD {
    using P = pair<int, int>;
    vector<vector<int>> g;
    vector<int> sz, in, out, rev, hs, par, dist;
    void dfs(int v, int p) {
        par[v] = p;
        sz[v] = 1;
        if (p != -1)
            dist[v] = dist[p] + 1;
        if (!g[v].empty() and g[v][0] == p)
            swap(g[v][0], g[v].back());
        for (auto &to : g[v])
            if (to != p) {
                dfs(to, v);
                sz[v] += sz[to];
                if (sz[g[v][0]] < sz[to])
                    swap(g[v][0], to);
            }
    }
    void dfs2(int v, int p, int &k) {
        in[v] = k++;
        rev[in[v]] = v;
        for (auto &to : g[v])
            if (to != p) {
                hs[to] = (g[v][0] == to ? hs[v] : to);
                dfs2(to, v, k);
            }
        out[v] = k;
    }
    HLD(int _n)
        : g(_n), sz(_n), in(_n), out(_n), rev(_n), hs(_n),
par(_n), dist(_n) {}
    void add_edge(int u, int v) {
        g[u].emplace_back(v);
        g[v].emplace_back(u);
    }
    void run(int rt = 0) {
        dfs(rt, -1);
        hs[rt] = rt;
        int k = 0;
        dfs2(rt, -1, k);
    }
    int lca(int u, int v) {
        for (;; v = par[hs[v]]) {
            if (in[u] > in[v])
                swap(u, v);
            if (hs[u] == hs[v])
                return u;
        }
    }
    vector<P> get(int u, int p, bool es = 0) {
        assert(in[p] <= in[u] and out[u] <= out[p]);
        vector<P> res;
        while (hs[u] != hs[p]) {
            res.push_back({in[hs[u]], in[u] + 1});
            u = par[hs[u]];
        }
        res.push_back({in[p] + es, in[u] + 1});
        return res;
    }
    int jump(int u, int v, int k) {
        if (k < 0)
            return -1;
        int g = lca(u, v);
        int d0 = dist[u] + dist[v] - dist[g] * 2;
        if (d0 < k)
            return -1;
        int st = u;
        if (dist[u] - dist[g] < k)
            st = v, k = d0 - k;
        for (;;) {
            int to = hs[st];
            if (in[st] - k >= in[to])
                return rev[in[st] - k];
            k -= in[st] - in[to] + 1;
            st = par[to];
        }
    }
};
```

hungarian.hpp

md5: d431c6

```
template <typename T, T MX> vector<int> Hungarian(int n,
vector<vector<T>> &A) {
    vector<int> u(n + 1), v(n + 1), q(n + 1, n), way(n + 1,
n);
    for (int i = 0; i < n; ++i) {
        q[n] = i;
        int j0 = n;
        vector<T> minv(n + 1, MX);
        vector<bool> used(n + 1, false);
        do {
            used[j0] = true;
            int i0 = q[j0], j1 = n;
```

```
            T delta = MX;
            for (int j = 0; j < n; ++j)
                if (!used[j]) {
                    T cur = A[i0][j] - u[i0] - v[j];
                    if (cur < minv[j])
                        minv[j] = cur, way[j] = j0;
                    if (minv[j] < delta)
                        delta = minv[j], j1 = j;
                }
            for (int j = 0; j <= n; ++j)
                if (used[j])
                    u[q[j]] += delta, v[j] -= delta;
                else
                    minv[j] -= delta;
            j0 = j1;
        } while (q[j0] != n);
        do {
            int j1 = way[j0];
            q[j0] = q[j1];
            j0 = j1;
        } while (j0 != n);
    }
    vector<int> p(n);
    rep(i, 0, n) p[q[i]] = i;
    return p;
}
```

linkcut.hpp

md5: 07a2b4

```
template <typename M, typename N, N (*f)(N, N)> struct LCT {
    struct Node {
        Node *lp = nullptr, *rp = nullptr, *par = nullptr;
        N val;
        M sum;
        int idx, sz = 1;
        bool rev = 0;
        Node() {}
        Node(int idx, N val) : idx(idx), val(val) {}
        void inv() {
            swap(lp, rp);
            sum.inv();
            rev ^= 1;
        }
        void eval() {
            if (rev) {
                if (lp)
                    lp->inv();
                if (rp)
                    rp->inv();
                rev = 0;
            }
        }
        void update() {
            sz = 1;
            if (lp)
                sz += lp->sz;
            if (rp)
                sz += rp->sz;
            sum.merge(val, lp ? lp->sum : M(), rp ? rp->sum
: M());
        }
        bool is_root() { return !par || (par->lp != this &&
par->rp != this); }
        void rotate() {
            Node *pp, *p, *c;
            p = par, pp = p->par;
            if (p->lp == this) {
                c = rp, rp = p, p->lp = c;
            } else {
                c = lp, lp = p, p->rp = c;
            }
            if (pp) {
                if (pp->lp == p)
                    pp->lp = this;
                if (pp->rp == p)
                    pp->rp = this;
            }
            par = pp, p->par = this;
            if (c)
                c->par = p;
            p->update(), update();
        }
        void splay() {
            eval();
            while (!is_root()) {
                Node *q = par;
                if (q->is_root()) {
                    q->eval(), eval();
                    rotate();
                } else {
                    Node *r = q->par;
                    r->eval(), q->eval(), eval();
                    if (r->lp == q) {
                        if (q->lp == this)
                            q->rotate();
                        else
                            rotate();
                        rotate();
                    } else {
                        if (q->rp == this)
                            q->rotate();
                        else
                            rotate();
                        rotate();
                    }
                }
            }
        }
    };
    LCT() {}
    Node *make(int idx, N val) { return new Node(idx, val);
}
    Node *expose(Node *v) {
        Node *pre = nullptr;
        for (Node *cur = v; cur; cur = cur->par) {
            cur->splay();
            if (cur->rp)
                cur->sum.add(cur->rp->sum);
            cur->rp = pre;
            if (cur->rp)
                cur->sum.sub(cur->rp->sum);
            cur->update();
            pre = cur;
        }
        v->splay();
        return pre;
    }
    void link(Node *c, Node *p) {
        evert(c);
        expose(p);
        c->par = p, p->rp = c;
        p->update();
    }
    void cut(Node *c, Node *p) {
        evert(p);
        expose(c);
        c->lp->par = nullptr;
        c->lp = nullptr;
        c->update();
    }
    void evert(Node *v) {
        expose(v);
        v->inv();
        v->eval();
    }
    Node *lca(Node *u, Node *v) {
        expose(u);
        return expose(v);
    }
    Node *root(Node *v) {
        expose(v);
        while (v->lp)
            v->eval(), v = v->lp;
        return v;
    }
    void update(Node *v, N x) {
        expose(v);
        v->val = f(v->val, x);
        v->update();
    }
    M &query(Node *u, Node *v) { // root = u -> v
        evert(u);
        expose(v);
        return v->sum;
    }
};

// using Key = ll;
// struct Monoid {
//     ll sum, psum, csum, light;
//     Monoid() : sum(0), psum(0), csum(0), light(0) {}
```

```
//      void inv() {
//          swap(psum, csum);
//      }
//      void merge(Key val, Monoid p, Monoid c) {
//          sum = p.sum + c.sum + val + light;
//          psum = p.psum + val + light;
//          csum = c.csum + val + light;
//      }
//      void add(Monoid v) {
//          light += v.sum;
//      }
//      void sub(Monoid v) {
//          light -= v.sum;
//      }
// };
// ll f(ll x, ll y) {
//      return x + y;
// }
// using V = LCT<Monoid, Key, f>::Node *;
```

lowlink.hpp

md5: 0dc367

```
struct LowLink {
    const int n;
    vector<vector<int>> g;
    vector<int> used, ord, low, id, arti;
    LowLink(const int &_n = 0)
        : n(_n), g(n), used(n, 0), ord(n, 0), low(n, 0),
id(n, -1) {}
    void add_edge(int u, int v) {
        g[u].emplace_back(v);
        g[v].emplace_back(u);
    }
    void dfs(int v, int p, int &k) {
        used[v] = 1;
        low[v] = ord[v] = k++;
        bool isarti = 0;
        int cnt = 0, sub = 0;
        for (auto &to : g[v]) {
            if (to == p and (++sub) <= 1)
                continue;
            if (!used[to]) {
                cnt++;
                dfs(to, v, k);
                chmin(low[v], low[to]);
                isarti |=
                    (p >= 0 && low[to] >= ord[v]); //
bridge: ord[v]<low[to]
            } else
                chmin(low[v], ord[to]);
        }
        isarti |= (p == -1 && cnt > 1);
        if (isarti)
            arti.push_back(v);
    }
    void dfs2(int v, int p, int &k) {
        if (p != -1 and ord[p] >= low[v])
            id[v] = id[p];
        else
            id[v] = k++;
        for (auto &to : g[v])
            if (id[to] == -1)
                dfs2(to, v, k);
    }
    int run() {
        int k = 0;
        rep(i, 0, n) if (!used[i]) dfs(i, -1, k);
        k = 0;
        rep(i, 0, n) if (id[i] == -1) dfs2(i, -1, k);
        return k;
    }
};
```

maxflow.hpp

md5: 6d9008

```
struct MaxFlow {
    struct Edge {
        int to, rev;
        ll cap;
    };
    int V;
    vector<vector<Edge>> G;
    vector<int> itr, level;
    using P = pair<int, int>;
    vector<P> es;

  public:
    MaxFlow() {}
    MaxFlow(int V) : V(V) { G.assign(V, vector<Edge>()); }
    int add_vertex() {
        G.push_back(vector<Edge>());
        return V++;
    }
    void add_edge(int from, int to, ll cap) {
        int fid = SZ(G[from]), tid = SZ(G[to]);
        if (from == to)
            tid++;
        es.push_back({from, fid});
        G[from].push_back({to, tid, cap});
        G[to].push_back({from, fid, 0});
    }
    struct Type {
        int from, to;
        ll cap, recap;
    };
    Type get_edge(int i) {
        auto [from, pos] = es[i];
        auto e = G[from][pos];
        auto re = G[e.to][e.rev];
        return Type{from, e.to, e.cap, re.cap};
    }
    void bfs(int s) {
        level.assign(V, -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (auto &e : G[v]) {
                if (e.cap > 0 && level[e.to] < 0) {
                    level[e.to] = level[v] + 1;
                    q.push(e.to);
                }
            }
        }
    }
    ll dfs(int v, int t, ll f) {
        if (v == t)
            return f;
        for (int &i = itr[v]; i < (int)G[v].size(); i++) {
            Edge &e = G[v][i];
            if (e.cap > 0 && level[v] < level[e.to]) {
                ll d = dfs(e.to, t, min(f, e.cap));
                if (d > 0) {
                    e.cap -= d, G[e.to][e.rev].cap += d;
                    return d;
                }
            }
        }
        return 0;
    }
    ll run(int s, int t) {
        ll ret = 0, f;
        while (bfs(s), level[t] >= 0) {
            itr.assign(V, 0);
            while ((f = dfs(s, t, INF)) > 0)
                ret += f;
        }
        return ret;
    }
};
```

mincostflow.hpp

md5: 0d3317

```
template <bool neg = 0, bool dag = 0> struct MinCostFlow {
    using P = pair<ll, int>;
    struct edge {
        int to;
        ll cap, cost;
        int rev;
        edge(int to, ll cap, ll cost, int rev)
            : to(to), cap(cap), cost(cost), rev(rev) {}
    };
    int n;
    vector<vector<edge>> g;
    vector<ll> dist, pot;
    vector<int> vpre, epre;
    MinCostFlow(int n) : n(n), g(n), dist(n), pot(n),
vpre(n), epre(n) {}
    void add_edge(int u, int v, ll cap, ll cost) {
```

```
        g[u].push_back({v, cap, cost, SZ(g[v])});
        g[v].push_back({u, 0, -cost, SZ(g[u]) - 1});
    }
    void bellmanford(int s) {
        pot.assign(n, INF);
        pot[s] = 0;
        for (;;) {
            bool upd = 0;
            rep(v, 0, n) if (pot[v] != INF) {
                for (auto &e : g[v]) {
                    if (e.cap > 0 and chmin(pot[e.to],
pot[v] + e.cost))
                        upd = 1;
                }
            }
            if (!upd)
                break;
        }
    }
    void bfs(int s) {
        vector<int> deg(n), tsort;
        queue<int> que;
        rep(v, 0, n) for (auto &e : g[v]) {
            if (e.cap > 0)
                deg[e.to]++;
        }
        rep(v, 0, n) if (deg[v] == 0) { que.push(v); }
        while (!que.empty()) {
            int v = que.front();
            tsort.push_back(v);
            que.pop();
            for (auto &e : g[v])
                if (e.cap > 0) {
                    deg[e.to]--;
                    if (deg[e.to] == 0)
                        que.push(e.to);
                }
        }
        pot.assign(n, INF);
        pot[s] = 0;
        for (auto &v : tsort)
            for (auto &e : g[v])
                if (e.cap > 0) {
                    chmin(pot[e.to], pot[v] + e.cost);
                }
    }
    ll run(int s, int t, ll flow) {
        ll ret = 0;
        if (neg) {
            if (dag)
                bfs(s);
            else
                bellmanford(s);
        }
        while (flow) {
            dist.assign(n, INF);
            priority_queue<P, vector<P>, greater<P>> pq;
            dist[s] = 0;
            pq.push({0, s});
            while (!pq.empty()) {
                auto [d, v] = pq.top();
                pq.pop();
                if (dist[v] != d)
                    continue;
                rep(j, 0, SZ(g[v])) {
                    auto e = g[v][j];
                    if (e.cap > 0 and
                        chmin(dist[e.to],
                              dist[v] + e.cost + pot[v] -
pot[e.to])) {
                        vpre[e.to] = v;
                        epre[e.to] = j;
                        pq.push({dist[e.to], e.to});
                    }
                }
            }
            if (dist[t] == INF)
                return -INF;
            rep(v, 0, n) {
                if (pot[v] == INF or dist[v] == INF)
                    pot[v] = INF;
                else
                    pot[v] += dist[v];
            }
            ll f = flow;
            for (int v = t; v != s; v = vpre[v])
                chmin(f, g[vpre[v]][epre[v]].cap);
            assert(f > 0);
            ret += pot[t] * f, flow -= f;
            for (int v = t; v != s; v = vpre[v]) {
                auto &e = g[vpre[v]][epre[v]];
                e.cap -= f;
                g[v][e.rev].cap += f;
            }
        }
        return ret;
    }
};
```

rerooting.hpp

md5: 864e01

```
template <typename M, typename N, M (*f)(M, N), M (*g)(M,
int), M (*h)(M, M),
          M (*e)()>
struct Rerooting {
    // f:action from edges, g:action from vertex, h:merge
two values
    using P = pair<int, N>;
    vector<vector<P>> G;
    vector<M> buf, ret;
    Rerooting(int n) : G(n) {}
    void add_edge(int u, int v, N w) {
        G[u].push_back({v, w});
        G[v].push_back({u, w});
    }
    void dfs1(int v, int p) {
        for (auto &[to, w] : G[v])
            if (to != p) {
                dfs1(to, v);
                buf[v] = h(buf[v], f(buf[to], w));
            }
        buf[v] = g(buf[v], v);
    }
    void dfs2(int v, int p, M pc) {
        buf[v] = e();
        vector<M> cs;
        for (auto &[to, w] : G[v])
            if (to != p) {
                cs.push_back(f(buf[to], w));
                buf[v] = h(buf[v], f(buf[to], w));
            }
        buf[v] = g(h(buf[v], pc), v);
        ret[v] = buf[v];
        cs.push_back(pc);
        int m = cs.size();
        vector<M> L(m), R(m);
        rep(i, 0, m) {
            L[i] = cs[i];
            if (i)
                L[i] = h(L[i], L[i - 1]);
        }
        for (int i = m - 1; i >= 0; i--) {
            R[i] = cs[i];
            if (i != m - 1)
                R[i] = h(R[i], R[i + 1]);
        }
        int id = 0;
        for (auto &[to, w] : G[v])
            if (to != p) {
                M nxt = e();
                if (id != 0)
                    nxt = h(nxt, L[id - 1]);
                if (id != m - 1)
                    nxt = h(nxt, R[id + 1]);
                M buf_v = buf[v], buf_to = buf[to];
                buf[v] = g(nxt, v);
                M add = f(buf[v], w);
                dfs2(to, v, add);
                buf[v] = buf_v, buf[to] = buf_to;
                id++;
            }
    }
    vector<M> run() {
        buf.assign(G.size(), e());
        ret.assign(G.size(), e());
        dfs1(0, -1);
        dfs2(0, -1, e());
        return ret;
    }
};
```

scc.hpp

md5: 1a667c

```
struct SCC {
    int n, m, cur;
    vector<vector<int>> g;
    vector<int> low, ord, id;
    SCC(int _n = 0)
        : n(_n), m(0), cur(0), g(_n), low(_n), ord(_n, -1),
id(_n) {}
    void resize(int _n) {
        n = _n;
        g.resize(n);
        low.resize(n);
        ord.resize(n, -1);
        id.resize(n);
    }
    void add_edge(int u, int v) { g[u].emplace_back(v); }
    void dfs(int v, vector<int> &used) {
        ord[v] = low[v] = cur++;
        used.emplace_back(v);
        for (auto &nxt : g[v]) {
            if (ord[nxt] == -1) {
                dfs(nxt, used);
                chmin(low[v], low[nxt]);
            } else {
                chmin(low[v], ord[nxt]);
            }
        }
        if (ord[v] == low[v]) {
            while (1) {
                int add = used.back();
                used.pop_back();
                ord[add] = n;
                id[add] = m;
                if (v == add)
                    break;
            }
            m++;
        }
    }
    void run() {
        vector<int> used;
        rep(v, 0, n) if (ord[v] == -1) dfs(v, used);
        for (auto &x : id)
            x = m - 1 - x;
    }
};
```

twosat.hpp

md5: eb6a31

```
struct TwoSat {
    int n;
    vector<bool> res;
    SCC graph;
    TwoSat(int _n) : n(_n), res(n), graph(n * 2) {}
    void add(int x) {
        n += x;
        res.resize(n);
        graph.resize(n * 2);
    }
    void either(int i, bool f, int j, bool g) {
        graph.add_edge(2 * i + (f ^ 1), 2 * j + g);
        graph.add_edge(2 * j + (g ^ 1), 2 * i + f);
    }
    void AtMostOne(vector<int> &a) {
        int offset = n;
        add(a.size());
        rep(i, 0, a.size()) either(a[i], 0, offset + i, 1);
        rep(i, 1, a.size()) either(offset + i - 1, 0, offset
+ i, 1);
        rep(i, 1, a.size()) either(a[i], 0, offset + i - 1,
0);
    }
    bool satisfiable() {
        graph.run();
        auto id = graph.id;
        rep(i, 0, n) {
            if (id[i * 2] == id[i * 2 + 1])
                return false;
            res[i] = id[i * 2] < id[i * 2 + 1];
        }
        return true;
    }
};
```

math

arbitrary.hpp

md5: e9912f

```
using M1 = fp<1045430273>;
using M2 = fp<1051721729>;
using M3 = fp<1053818881>;
const M2 r_12 = M2(M1::get_mod()).inv();
const M3 r_13 = M3(M1::get_mod()).inv();
const M3 r_23 = M3(M2::get_mod()).inv();
const M3 r_1323 = r_13 * r_23;
template <typename T>
vector<T> ArbitraryMult(const vector<T> &a, const vector<T>
&b, bool same = 0) {
    if (a.empty() or b.empty())
        return vector<T>();
    int n = a.size() + b.size() - 1;
    vector<T> res(n);
    if (min(a.size(), b.size()) <= 60) {
        rep(i, 0, a.size()) rep(j, 0, b.size()) res[i + j]
+= a[i] * b[j];
        return res;
    }
    vector<int> vals[3];
    vector<int> aa(a.size()), bb(b.size());
    rep(i, 0, a.size()) aa[i] = a[i].v;
    rep(i, 0, b.size()) bb[i] = b[i].v;
    vector<M1> a1(ALL(aa)), b1(ALL(bb)), c1 = mult<M1, 3>
(a1, b1, same);
    vector<M2> a2(ALL(aa)), b2(ALL(bb)), c2 = mult<M2, 6>
(a2, b2, same);
    vector<M3> a3(ALL(aa)), b3(ALL(bb)), c3 = mult<M3, 7>
(a3, b3, same);
    for (M1 x : c1)
        vals[0].push_back(x.v);
    for (M2 x : c2)
        vals[1].push_back(x.v);
    for (M3 x : c3)
        vals[2].push_back(x.v);
    T w1(M1::get_mod()), w2 = w1 * T(M2::get_mod());
    rep(i, 0, n) {
        ll p = vals[0][i];
        ll q = (vals[1][i] + M2::get_mod() - p) * r_12.v %
M2::get_mod();
        ll r = ((vals[2][i] + M3::get_mod() - p) * r_1323.v
+
                (M3::get_mod() - q) * r_23.v) %
               M3::get_mod();
        res[i] = (p + q * w1.v + r * w2.v);
    }
    return res;
}
```

berlekamp.hpp

md5: 883339

```
template <typename T> vector<T> BerlekampMassey(vector<T>
&a) {
    int n = a.size();
    T d = 1;
    vector<T> b(1), c(1);
    b[0] = c[0] = 1;
    rep(j, 1, n + 1) {
        int l = c.size(), m = b.size();
        T x = 0;
        rep(i, 0, l) x += c[i] * a[j - l + i];
        b.push_back(0);
        m++;
        if (x == 0)
            continue;
        T coeff = -x / d;
        if (l < m) {
            auto tmp = c;
            c.insert(c.begin(), m - l, 0);
            rep(i, 0, m) c[m - 1 - i] += coeff * b[m - 1 -
i];
            b = tmp;
            d = x;
        } else
            rep(i, 0, m) c[l - 1 - i] += coeff * b[m - 1 -
i];
    }
    return c;
}
```

bitwise.hpp

md5: 442b10

```
template <typename T> void zeta(vector<T> &a) {
    int n = __lg(a.size());
    rep(k, 0, n) rep(mask, 0, 1 << n) {
        if (mask >> k & 1)
            a[mask] += a[mask ^ (1 << k)];
    }
}
template <typename T> void mobius(vector<T> &a) {
    int n = __lg(a.size());
    rep(k, 0, n) rep(mask, 0, 1 << n) {
        if (mask >> k & 1)
            a[mask] -= a[mask ^ (1 << k)];
    }
}
template <typename T> void fwt(vector<T> &a) {
    int n = __lg(a.size());
    rep(k, 0, n) rep(mask, 0, 1 << n) {
        if (!(mask >> k & 1)) {
            T x = a[mask], y = a[mask | (1 << k)];
            a[mask] = x + y, a[mask | (1 << k)] = x - y;
        }
    }
}
```

bostanmori.hpp

md5: 3905c1

```
template <typename T> T nth(Poly<T> p, Poly<T> q, ll n) {
    while (n) {
        Poly<T> base(q), np, nq;
        for (int i = 1; i < (int)q.size(); i += 2)
            base[i] = -base[i];
        p *= base;
        q *= base;
        for (int i = n & 1; i < (int)p.size(); i += 2)
            np.emplace_back(p[i]);
        for (int i = 0; i < (int)q.size(); i += 2)
            nq.emplace_back(q[i]);
        swap(p, np);
        swap(q, nq);
        n >>= 1;
    }
    return p[0] / q[0];
}
```

comb.hpp

md5: 37e3c5

```
template <typename T> struct Comb {
    int n;
    vector<T> fact, ifact;
    Comb() {}
    Comb(int n) : fact(n + 1), ifact(n + 1) {
        fact[0] = 1;
        rep(i, 1, n + 1) fact[i] = fact[i - 1] * i;
        ifact[n] = fact[n].inv();
        rrep(i, 0, n) ifact[i] = ifact[i + 1] * (i + 1);
    }
    T Fact(int n, bool inv = 0) {
        if (n < 0)
            return 0;
        return (inv ? ifact[n] : fact[n]);
    }
    T nCr(int n, int r, bool inv = 0) {
        if (n < 0 or r < 0 or n - r < 0)
            return 0;
        return Fact(n, inv) * Fact(r, !inv) * Fact(n - r,
!inv);
    }
};
```

crt.hpp

md5: 3ee261

```
ll extgcd(ll a, ll b, ll &p, ll &q) {
    if (b == 0) {
        p = 1;
        q = 0;
        return a;
    }
    ll d = extgcd(b, a % b, q, p);
    q -= a / b * p;
    return d;
}
pair<ll, ll> crt(const vector<ll> &vs, const vector<ll> &ms)
{
    ll V = vs[0], M = ms[0];
    rep(i, 1, vs.size()) {
        ll p, q, v = vs[i], m = ms[i];
        if (M < m)
            swap(M, m), swap(V, v);
        ll d = extgcd(M, m, p, q);
        if ((v - V) % d != 0)
            return {0, -1};
        ll md = m / d, tmp = (v - V) / d % md * p % md;
        V += M * tmp;
        M *= md;
    }
    V = (V % M + M) % M;
    return {V, M};
}
```

enumquotient.hpp

md5: 4611f3

```
template <typename T> vector<pair<T, T>> EnumQuotients(T n)
{
    T SQ = sqrtl(n);
    while (SQ * SQ + SQ <= n)
        SQ++;
    T m = n / SQ;
    vector<pair<T, T>> ret;
    rep(i, 1, m + 1) ret.emplace_back(n / (i + 1) + 1, n / i
+ 1);
    rrep(i, 1, SQ) ret.emplace_back(i, i + 1);
    return ret;
}
```

extgcd.hpp

md5: d1247c

```
ll extgcd(ll a, ll b, ll &p, ll &q) { //
argmin{|p|+|q|:ap+bq=gcd(a,b)}
    if (b == 0) {
        p = 1, q = 0;
        return a;
    }
    ll d = extgcd(b, a % b, q, p);
    q -= a / b * p;
    return d;
}
ll minv(ll a, ll m) {
    ll y, z;
    extgcd(a, m, y, z);
    return (y % m + m) % m;
}
```

floorsum.hpp

md5: 054fd5

```
template <typename T = ll>
T FloorSum(ll n, ll a, ll b, ll m) { // sum_{k=0}^{n-1}
[(a*k+b)/m]
    T res = 0;
    if (a >= m)
        res += T(a / m) * n * (n - 1) / 2, a -= a / m * m;
    else if (a < 0)
        res -= T((-a + m - 1) / m) * n * (n - 1) / 2,
            a += ((-a + m - 1) / m) * m;
    if (b >= m)
        res += (b / m) * n, b -= b / m * m;
    else if (b < 0)
        res -= ((-b + m - 1) / m) * n, b += ((-b + m - 1) /
m) * m;
    while (1) {
        ll y_max = a * n + b;
        if (y_max < m)
            break;
        n = y_max / m;
        b = y_max % m;
        res += (n * (n - 1) / 2) * (m / a) + n * (b / a);
        swap(m, a);
        a = a % m;
        b = b % m;
    }
    return res;
}
```

fps.hpp

md5: 9ef798

```
template <typename T> struct Poly : vector<T> {
    Poly(int n = 0) { this->assign(n, T()); }
```

```
    Poly(const initializer_list<T> f) : vector<T>::vector(f)
{}
    Poly(const vector<T> &f) { this->assign(ALL(f)); }
    T eval(const T &x) {
        T res;
        for (int i = this->size() - 1; i >= 0; i--)
            res *= x, res += this->at(i);
        return res;
    }
    Poly rev() const {
        Poly res = *this;
        reverse(ALL(res));
        return res;
    }
    void shrink() {
        while (!this->empty() and this->back() == 0)
            this->pop_back();
    }
    Poly inv() const {
        assert(this->front() != 0);
        const int n = this->size();
        Poly res(1);
        res.front() = T(1) / this->front();
        for (int k = 1; k < n; k <<= 1) {
            Poly g = res, h = *this;
            h.resize(k * 2);
            res.resize(k * 2);
            g = (g.square() * h);
            g.resize(k * 2);
            rep(i, k, min(k * 2, n)) res[i] -= g[i];
        }
        res.resize(n);
        return res;
    }
    Poly square() const { return Poly(mult(*this, *this,
1)); }
    Poly operator-() const { return Poly() - *this; }
    Poly operator+(const Poly &g) const { return Poly(*this)
+= g; }
    Poly operator+(const T &g) const { return Poly(*this) +=
g; }
    Poly operator-(const Poly &g) const { return Poly(*this)
-= g; }
    Poly operator-(const T &g) const { return Poly(*this) -=
g; }
    Poly operator*(const Poly &g) const { return Poly(*this)
*= g; }
    Poly operator*(const T &g) const { return Poly(*this) *=
g; }
    Poly operator/(const Poly &g) const { return Poly(*this)
/= g; }
    Poly operator%(const Poly &g) const { return Poly(*this)
%= g; }
    pair<Poly, Poly> divmod(const Poly &g) const {
        Poly q = *this / g, r = *this - g * q;
        r.shrink();
        return {q, r};
    }
    Poly &operator+=(const Poly &g) {
        if (g.size() > this->size())
            this->resize(g.size());
        rep(i, 0, g.size()) { (*this)[i] += g[i]; }
        return *this;
    }
    Poly &operator+=(const T &g) {
        if (this->empty())
            this->push_back(0);
        (*this)[0] += g;
        return *this;
    }
    Poly &operator-=(const Poly &g) {
        if (g.size() > this->size())
            this->resize(g.size());
        rep(i, 0, g.size()) { (*this)[i] -= g[i]; }
        return *this;
    }
    Poly &operator-=(const T &g) {
        if (this->empty())
            this->push_back(0);
        (*this)[0] -= g;
        return *this;
    }
    Poly &operator*=(const Poly &g) {
        *this = mult(*this, g, 0);
        return *this;
    }
    Poly &operator*=(const T &g) {
        rep(i, 0, this->size())(*this)[i] *= g;
        return *this;
    }
    Poly &operator/=(const Poly &g) {
        if (g.size() > this->size()) {
            this->clear();
            return *this;
        }
        Poly g2 = g;
        reverse(ALL(*this));
        reverse(ALL(g2));
        int n = this->size() - g2.size() + 1;
        this->resize(n);
        g2.resize(n);
        *this *= g2.inv();
        this->resize(n);
        reverse(ALL(*this));
        shrink();
        return *this;
    }
    Poly &operator%=(const Poly &g) {
        *this -= *this / g * g;
        shrink();
        return *this;
    }
    Poly diff() const {
        Poly res(this->size() - 1);
        rep(i, 0, res.size()) res[i] = (*this)[i + 1] * (i +
1);
        return res;
    }
    Poly inte() const {
        Poly res(this->size() + 1);
        for (int i = res.size() - 1; i; i--)
            res[i] = (*this)[i - 1] / i;
        return res;
    }
    Poly log() const {
        assert(this->front() == 1);
        const int n = this->size();
        Poly res = diff() * inv();
        res = res.inte();
        res.resize(n);
        return res;
    }
    Poly exp() const {
        assert(this->front() == 0);
        const int n = this->size();
        Poly res(1), g(1);
        res.front() = g.front() = 1;
        for (int k = 1; k < n; k <<= 1) {
            g = (g + g - g.square() * res);
            g.resize(k);
            Poly q = *this;
            q.resize(k);
            q = q.diff();
            Poly w = (q + g * (res.diff() - res * q)), t =
*this;
            w.resize(k * 2 - 1);
            t.resize(k * 2);
            res = (res + res * (t - w.inte()));
            res.resize(k * 2);
        }
        res.resize(n);
        return res;
    }
    Poly shift(const int &c) const {
        const int n = this->size();
        Poly res = *this, g(n);
        g[0] = 1;
        rep(i, 1, n) g[i] = g[i - 1] * c / i;
        vector<T> fact(n, 1);
        rep(i, 0, n) {
            if (i)
                fact[i] = fact[i - 1] * i;
            res[i] *= fact[i];
        }
        res = res.rev();
        res *= g;
        res.resize(n);
        res = res.rev();
        rep(i, 0, n) res[i] /= fact[i];
        return res;
    }
```

```
    Poly pow(ll t) {
        if (t == 0) {
            Poly res(this->size());
            res[0] = 1;
            return res;
        }
        int n = this->size(), k = 0;
        while (k < n and (*this)[k] == 0)
            k++;
        Poly res(n);
        if (__int128_t(t) * k >= n)
            return res;
        n -= t * k;
        Poly g(n);
        T c = (*this)[k], ic = c.inv();
        rep(i, 0, n) g[i] = (*this)[i + k] * ic;
        g = g.log();
        for (auto &x : g)
            x *= t;
        g = g.exp();
        c = c.pow(t);
        rep(i, 0, n) res[i + t * k] = g[i] * c;
        return res;
    }
    vector<T> mult(const vector<T> &a, const vector<T> &b,
bool same) const;
};
// template<>vector<Fp> Poly<Fp>::mult(const vector<Fp>&
a,const vector<Fp>&
// b,bool same)const{return mult(a,b);}
```

getprimroot.hpp

md5: f1fd47

```
ll getPrimitiveRoot(ll p) {
    vector<ll> ps = Pollard(p - 1);
    for (;;) {
        ll x = RNG(1, p - 1);
        for (auto &q : ps) {
            if (mpow(x, (p - 1) / q, p) == 1)
                goto fail;
        }
        return x;
    fail:;
    }
    assert(0);
}
```

miller.hpp

md5: 122cd7

```
bool Miller(ll n) {
    if (n < 2 or (n & 1) == 0)
        return (n == 2);
    ll d = n - 1;
    while ((d & 1) == 0)
        d >>= 1;
    vector<ll> seeds;
    if (n < (1 << 30))
        seeds = {2, 7, 61};
    else
        seeds = {2, 325, 9375, 28178, 450775, 9780504};
    for (auto &x : seeds) {
        if (n <= x)
            break;
        ll t = d;
        ll y = mpow(x, t, n);
        while (t != n - 1 and y != 1 and y != n - 1) {
            y *= y;
            t <<= 1;
        }
        if (y != n - 1 and (t & 1) == 0)
            return 0;
    }
    return 1;
}
```

modint.hpp

md5: 796a63

```
template <unsigned mod = 1000000007> struct fp {
    unsigned v;
    static constexpr int get_mod() { return mod; }
    constexpr unsigned inv() const {
        assert(v != 0);
        int x = v, y = mod, p = 1, q = 0, t = 0, tmp = 0;
        while (y > 0) {
            t = x / y;
            x -= t * y, p -= t * q;
            tmp = x, x = y, y = tmp;
            tmp = p, p = q, q = tmp;
        }
        if (p < 0)
            p += mod;
        return p;
    }
    constexpr fp(ll x = 0) : v(x >= 0 ? x % mod : (mod - (-
x) % mod) % mod) {}
    fp operator-() const { return fp() - *this; }
    fp pow(ull t) {
        fp res = 1, b = *this;
        while (t) {
            if (t & 1)
                res *= b;
            b *= b;
            t >>= 1;
        }
        return res;
    }
    fp &operator+=(const fp &x) {
        if ((v += x.v) >= mod)
            v -= mod;
        return *this;
    }
    fp &operator-=(const fp &x) {
        if ((v += mod - x.v) >= mod)
            v -= mod;
        return *this;
    }
    fp &operator*=(const fp &x) {
        v = ull(v) * x.v % mod;
        return *this;
    }
    fp &operator/=(const fp &x) {
        v = ull(v) * x.inv() % mod;
        return *this;
    }
    fp operator+(const fp &x) const { return fp(*this) += x;
}
    fp operator-(const fp &x) const { return fp(*this) -= x;
}
    fp operator*(const fp &x) const { return fp(*this) *= x;
}
    fp operator/(const fp &x) const { return fp(*this) /= x;
}
    bool operator==(const fp &x) const { return v == x.v; }
    bool operator!=(const fp &x) const { return v != x.v; }
    friend istream &operator>>(istream &is, fp &x) { return
is >> x.v; }
    friend ostream &operator<<(ostream &os, const fp &x) {
return os << x.v; }
};
```

ntt.hpp

md5: 398f39

```
template <typename T> void NTT(vector<T> &a, bool type) {
    int n = SZ(a), s = 0;
    while ((1 << s) < n)
        s++;
    assert((1 << s) == n);
    static T g = 2;
    static vector<T> pw, ipw;
    while (g.pow((T::get_mod() - 1) >> 1) == 1) {
        g += 1;
    }
    while (SZ(pw) <= s) {
        pw.push_back(g.pow((T(-1).v) >> SZ(pw)));
        ipw.push_back(pw.back().inv());
    }
    vector<T> b(n);
    rep(i, 1, s + 1) {
        int wid = 1 << (s - i);
        T base = type ? ipw[i] : pw[i], rot = 1;
        for (int y = 0; y < n / 2; y += wid) {
            for (int x = 0; x < wid; x++) {
                auto L = a[y << 1 | x];
                auto R = a[y << 1 | x | wid] * rot;
                b[y | x] = L + R;
                b[y | x | n >> 1] = L - R;
            }
            rot *= base;
        }
        swap(a, b);
```

```
    }
}

template <typename T>
vector<T> mult(vector<T> &a, vector<T> &b, bool same = 0) {
    if (a.empty() or b.empty())
        return {};
    int n = SZ(a), m = SZ(b), deg = 1;
    while (deg < n + m - 1)
        deg <<= 1;
    vector<T> ret(deg);
    rep(i, 0, n) ret[i] = a[i];
    NTT(ret, 0);
    if (same) {
        rep(i, 0, deg) ret[i] *= ret[i];
    } else {
        vector<T> c(deg);
        rep(i, 0, m) c[i] = b[i];
        NTT(c, 0);
        rep(i, 0, deg) ret[i] *= c[i];
    }
    NTT(ret, 1);
    ret.resize(n + m - 1);
    T e = T(deg).inv();
    for (auto &x : ret)
        x *= e;
    return ret;
}
```

pollard.hpp

md5: e38667

```
vector<ll> Pollard(ll n) {
    if (n <= 1)
        return {};
    if (Miller(n))
        return {n};
    if ((n & 1) == 0) {
        vector<ll> v = Pollard(n >> 1);
        v.push_back(2);
        return v;
    }
    for (ll x = 2, y = 2, d;;) {
        ll c = RNG(2LL, n - 1);
        do {
            x = (__int128_t(x) * x + c) % n;
            y = (__int128_t(y) * y + c) % n;
            y = (__int128_t(y) * y + c) % n;
            d = __gcd(x - y + n, n);
        } while (d == 1);
        if (d < n) {
            vector<ll> lb = Pollard(d), rb = Pollard(n / d);
            lb.insert(lb.end(), ALL(rb));
            return lb;
        }
    }
}
```

geometry

argsort.hpp

md5: 4999d2

```
bool cmp(const Point &a, const Point &b) {
    auto sub = [&](Point a) {
        return (a.Y < 0 ? -1 : (a.Y == 0 && a.X >= 0 ? 0 :
1));
    };
    if (sub(a) != sub(b))
        return sub(a) < sub(b);
    return a.Y * b.X < a.X * b.Y;
}
```

ccw.hpp

md5: f53c7d

```
int ccw(const Point &a, Point b, Point c) {
    b -= a;
    c -= a;
    if (cross(b, c) > eps)
        return 1; // ccw
    if (cross(b, c) < -eps)
        return -1; // cw
    if (dot(b, c) < 0)
        return 2; // c,a,b
    if (norm(b) < norm(c))
        return -2; // a,b,c
    return 0;      // a,c,b
}
```

closest.hpp

md5: 2c1282

```
T Closest(Poly &a) {
    int n = a.size();
    if (n <= 1)
        return 0;
    sort(ALL(a), [&](Point a, Point b) {
        return (eq(a.X, b.X) ? a.Y < b.Y : a.X < b.X);
    });
    Poly buf(n);
    auto rec = [&](auto self, int lb, int rb) -> T {
        if (rb - lb <= 1)
            return (T)INF;
        int mid = (lb + rb) >> 1;
        auto x = a[mid].X;
        T res = min(self(self, lb, mid), self(self, mid,
rb));
        inplace_merge(a.begin() + lb, a.begin() + mid,
a.begin() + rb,
                      [&](auto p, auto q) { return p.Y <
q.Y; });
        int ptr = 0;
        rep(i, lb, rb) {
            if (abs(a[i].X - x) >= res)
                continue;
            rep(j, 0, ptr) {
                auto sub = a[i] - buf[ptr - 1 - j];
                if (sub.Y >= res)
                    break;
                chmin(res, abs(sub));
            }
            buf[ptr++] = a[i];
        }
        return res;
    };
    return rec(rec, 0, n);
}
```

convexhull.hpp

md5: 6e884c

```
Poly ConvexHull(Poly &a) {
    int n = a.size(), k = 0;
    sort(ALL(a), [](const Point &p, const Point &q) {
        return (eq(p.Y, q.Y) ? p.X < q.X : p.Y < q.Y);
    });
    Poly res(n * 2);
    for (int i = 0; i < n; res[k++] = a[i++]) {
        while (k >= 2 and
                cross(res[k - 1] - res[k - 2], a[i] - res[k -
1]) < -eps)
            k--;
    }
    for (int i = n - 2, t = k + 1; i >= 0; res[k++] = a[i-
-]) {
        while (k >= t and
                cross(res[k - 1] - res[k - 2], a[i] - res[k -
1]) < -eps)
            k--;
    }
    res.resize(k - 1);
    return res;
}
```

enclosing.hpp

md5: b59e0a

```
Circle Enclosing(vector<Point> ps) {
    Random::shuffle(ALL(ps));
    auto make2 = [&](Point p, Point q) {
        return Circle((p + q) * .5, abs(p - q) * .5);
    };
    Circle ret = make2(ps[0], ps[1]);
    rep(i, 2, SZ(ps)) if (abs(ret.p - ps[i]) > ret.r + eps)
{
        ret = make2(ps[0], ps[i]);
        rep(j, 1, i) if (abs(ret.p - ps[j]) > ret.r + eps) {
            ret = make2(ps[i], ps[j]);
            rep(k, 0, j) if (abs(ret.p - ps[k]) > ret.r +
eps) {
                ret = Circumcircle(ps[i], ps[j], ps[k]);
            }
        }
```

```
    }
    return ret;
}
```

farthest.hpp

md5: 299b57

```
T farthest(const Poly &a) {
    int n = a.size();
    int x = 0, y = 0;
    rep(i, 1, n) {
        if (a[i].Y > a[x].Y)
            x = i;
        if (a[i].Y < a[y].Y)
            y = i;
    }
    T res = abs(a[x] - a[y]);
    int i = x, j = y;
    do {
        if (cross(a[(i + 1) % n] - a[i], a[(j + 1) % n] -
a[j]) < 0)
            i = (i + 1) % n;
        else
            j = (j + 1) % n;
        chmax(res, abs(a[i] - a[j]));
    } while (i != x or j != y);
    return res;
}
```

geometry.hpp

md5: 872a68

```
using T = double;
const T eps = 1e-12;
using Point = complex<T>;
using Poly = vector<Point>;
#define X real()
#define Y imag()
inline bool eq(const T &a, const T &b) { return fabs(a - b)
< eps; }
istream &operator>>(istream &is, Point &p) {
    T x, y;
    is >> x >> y;
    p = Point(x, y);
    return is;
}

struct Line {
    Point a, b, dir;
    Line() {}
    Line(Point _a, Point _b) : a(_a), b(_b), dir(b - a) {}
    Line(T A, T B, T C) {
        if (eq(A, .0)) {
            a = Point(0, C / B), b = Point(1 / C / B);
        } else if (eq(B, .0)) {
            a = Point(C / A, 0), b = Point(C / A, 1);
        } else {
            a = Point(0, C / B), b = Point(C / A, 0);
        }
    }
};
struct Segment : Line {
    Segment() {}
    Segment(Point _a, Point _b) : Line(_a, _b) {}
};
struct Circle {
    Point p;
    T r;
    Circle() {}
    Circle(Point _p, T _r) : p(_p), r(_r) {}
};

Point unit(const Point &a) { return a / abs(a); }
T dot(const Point &a, const Point &b) { return a.X * b.X +
a.Y * b.Y; }
T cross(const Point &a, const Point &b) { return a.X * b.Y -
a.Y * b.X; }
Point rot(const Point &a, const T &theta) {
    return Point(cos(theta) * a.X - sin(theta) * a.Y,
                 sin(theta) * a.X + cos(theta) * a.Y);
}
Point rot90(const Point &a) { return Point(-a.Y, a.X); }
T arg(const Point &a, const Point &b, const Point &c) {
    double ret = acos(dot(a - b, c - b) / abs(a - b) / abs(c
- b));
    if (cross(a - b, c - b) < 0)
        ret = -ret;
    return ret;
}

Point Projection(const Line &l, const Point &p) {
    T t = dot(p - l.a, l.a - l.b) / norm(l.a - l.b);
    return l.a + (l.a - l.b) * t;
}
Point Reflection(const Line &l, const Point &p) {
    return p + (Projection(l, p) - p) * 2.;
}

bool isOrthogonal(const Line &a, const Line &b) {
    return eq(dot(a.b - a.a, b.b - b.a), .0);
}
bool isParallel(const Line &a, const Line &b) {
    return eq(cross(a.b - a.a, b.b - b.a), .0);
}
bool isIntersect(const Segment &a, const Segment &b) {
    return ccw(a.a, a.b, b.a) * ccw(a.a, a.b, b.b) <= 0 and
           ccw(b.a, b.b, a.a) * ccw(b.a, b.b, a.b) <= 0;
}
int isIntersect(const Circle &a, const Circle &b) {
    T d = abs(a.p - b.p);
    if (d > a.r + b.r + eps)
        return 4;
    if (eq(d, a.r + b.r))
        return 3;
    if (eq(d, abs(a.r - b.r)))
        return 1;
    if (d < abs(a.r - b.r) - eps)
        return 0;
    return 2;
}

T Dist(const Line &a, const Point &b) {
    Point c = Projection(a, b);
    return abs(c - b);
}
T Dist(const Segment &a, const Point &b) {
    if (dot(a.b - a.a, b - a.a) < eps)
        return abs(b - a.a);
    if (dot(a.a - a.b, b - a.b) < eps)
        return abs(b - a.b);
    return abs(cross(a.b - a.a, b - a.a)) / abs(a.b - a.a);
}
T Dist(const Segment &a, const Segment &b) {
    if (isIntersect(a, b))
        return .0;
    T res = min({Dist(a, b.a), Dist(a, b.b), Dist(b, a.a),
Dist(b, a.b)});
    return res;
}

Point Intersection(const Line &a, const Line &b) {
    T d1 = cross(a.b - a.a, b.b - b.a);
    T d2 = cross(a.b - a.a, a.b - b.a);
    if (eq(d1, 0) and eq(d2, 0))
        return b.a;
    return b.a + (b.b - b.a) * (d2 / d1);
}
vector<Point> Intersection(const Circle &a, const Line &b) {
    vector<Point> res;
    T d = Dist(b, a.p);
    if (d > a.r + eps)
        return res;
    Point h = Projection(b, a.p);
    if (eq(d, a.r)) {
        res.push_back(h);
        return res;
    }
    Point e = unit(b.b - b.a);
    T ph = sqrt(a.r * a.r - d * d);
    res.push_back(h - e * ph);
    res.push_back(h + e * ph);
    return res;
}
vector<Point> Intersection(const Circle &a, const Circle &b)
{
    vector<Point> res;
    int mode = isIntersect(a, b);
    T d = abs(a.p - b.p);
    if (mode == 4 or mode == 0)
        return res;
    if (mode == 3) {
        T t = a.r / (a.r + b.r);
```

```
        res.push_back(a.p + (b.p - a.p) * t);
        return res;
    }
    if (mode == 1) {
        if (b.r < a.r - eps) {
            res.push_back(a.p + (b.p - a.p) * (a.r / d));
        } else {
            res.push_back(b.p + (a.p - b.p) * (b.r / d));
        }
        return res;
    }
    T rc = (a.r * a.r + d * d - b.r * b.r) / d / 2.;
    T rs = sqrt(a.r * a.r - rc * rc);
    if (a.r - abs(rc) < eps)
        rs = 0;
    Point e = unit(b.p - a.p);
    res.push_back(a.p + rc * e + rs * e * Point(0, 1));
    res.push_back(a.p + rc * e + rs * e * Point(0, -1));
    return res;
}

T Area(const Poly &a) {
    T res = 0;
    int n = a.size();
    rep(i, 0, n) res += cross(a[i], a[(i + 1) % n]);
    return fabs(res / 2.);
}
T Area(const Poly &a, const Circle &b) {
    int n = a.size();
    if (n < 3)
        return .0;
    auto rec = [&](auto self, const Circle &c, const Point
&p1,
                   const Point &p2) {
        Point va = c.p - p1, vb = c.p - p2;
        T f = cross(va, vb), res = .0;
        if (eq(f, .0))
            return res;
        if (max(abs(va), abs(vb)) < c.r + eps)
            return f;
        if (Dist(Segment(p1, p2), c.p) > c.r - eps)
            return c.r * c.r * arg(vb * conj(va));
        auto u = Intersection(c, Segment(p1, p2));
        Poly sub;
        sub.push_back(p1);
        for (auto &x : u)
            sub.push_back(x);
        sub.push_back(p2);
        rep(i, 0, sub.size() - 1) res += self(self, c,
sub[i], sub[i + 1]);
        return res;
    };
    T res = .0;
    rep(i, 0, n) res += rec(rec, b, a[i], a[(i + 1) % n]);
    return fabs(res / 2.);
}
T Area(const Circle &a, const Circle &b) {
    T d = abs(a.p - b.p);
    if (d >= a.r + b.r - eps)
        return .0;
    if (d <= abs(a.r - b.r) + eps) {
        T r = min(a.r, b.r);
        return M_PI * r * r;
    }
    T ath = acos((a.r * a.r + d * d - b.r * b.r) / d / a.r /
2.);
    T res = a.r * a.r * (ath - sin(ath * 2) / 2.);
    T bth = acos((b.r * b.r + d * d - a.r * a.r) / d / b.r /
2.);
    res += b.r * b.r * (bth - sin(bth * 2) / 2.);
    return fabs(res);
}

Circle Incircle(const Point &a, const Point &b, const Point
&c) {
    T A = abs(b - c), B = abs(c - a), C = abs(a - b);
    Point p(A * a.X + B * b.X + C * c.X, A * a.Y + B * b.Y +
C * c.Y);
    p /= (A + B + C);
    T r = Dist(Line(a, b), p);
    return Circle(p, r);
}
Circle Circumcircle(const Point &a, const Point &b, const
Point &c) {
    Line l1((a + b) / 2., (a + b) / 2. + (b - a) * Point(0,
1));
    Line l2((b + c) / 2., (b + c) / 2. + (c - b) * Point(0,
1));
    Point p = Intersection(l1, l2);
    return Circle(p, abs(p - a));
}

Poly tangent(const Point &a, const Circle &b) {
    return Intersection(b, Circle(a, sqrt(norm(b.p - a) -
b.r * b.r)));
}
vector<Line> tangent(const Circle &a, const Circle &b) {
    vector<Line> res;
    T d = abs(a.p - b.p);
    if (eq(d, 0))
        return res;
    Point u = unit(b.p - a.p);
    Point v = u * Point(0, 1);
    for (int t : {-1, 1}) {
        T h = (a.r + b.r * t) / d;
        if (eq(h * h, 1)) {
            res.push_back(Line(a.p + (h > 0 ? u : -u) * a.r,
                               a.p + (h > 0 ? u : -u) * a.r
+ v));
        } else if (1 > h * h) {
            Point U = u * h, V = v * sqrt(1 - h * h);
            res.push_back(Line(a.p + (U + V) * a.r, b.p - (U
+ V) * (b.r * t)));
            res.push_back(Line(a.p + (U - V) * a.r, b.p - (U
- V) * (b.r * t)));
        }
    }
    return res;
}
```

halfplane.hpp

md5: 57d4c6

```
Poly HalfplaneIntersection(vector<Line> &H) {
    sort(ALL(H), [&](Line &l1, Line &l2) { return
cmp(l1.dir, l2.dir); });
    auto outside = [&](Line &L, Point p) -> bool {
        return cross(L.dir, p - L.a) < -eps;
    };
    deque<Line> deq;
    int sz = 0;
    rep(i, 0, SZ(H)) {
        while (sz > 1 and
               outside(H[i], Intersection(deq[sz - 1],
deq[sz - 2]))) {
            deq.pop_back();
            sz--;
        }
        while (sz > 1 and outside(H[i], Intersection(deq[0],
deq[1]))) {
            deq.pop_front();
            sz--;
        }
        if (sz > 0 and fabs(cross(H[i].dir, deq[sz -
1].dir)) < eps) {
            if (dot(H[i].dir, deq[sz - 1].dir) < 0) {
                return {};
            }
            if (outside(H[i], deq[sz - 1].a)) {
                deq.pop_back();
                sz--;
            } else
                continue;
        }
        deq.push_back(H[i]);
        sz++;
    }
    while (sz > 2 and outside(deq[0], Intersection(deq[sz -
1], deq[sz - 2]))) {
        deq.pop_back();
        sz--;
    }
    while (sz > 2 and outside(deq[sz - 1],
Intersection(deq[0], deq[1]))) {
        deq.pop_front();
        sz--;
    }
    if (sz < 3)
        return {};
    deq.push_back(deq.front());
    Poly ret;
    rep(i, 0, sz) ret.push_back(Intersection(deq[i], deq[i +
```

```
1]));
    return ret;
}
```

iscontain.hpp

md5: 8404a4

```
int isContained(const Poly &a,
                const Point &b) { // 0:not contain,1:on
edge,2:contain
    bool res = 0;
    int n = a.size();
    rep(i, 0, n) {
        Point p = a[i] - b, q = a[(i + 1) % n] - b;
        if (p.Y > q.Y)
            swap(p, q);
        if (p.Y < eps and eps < q.Y and cross(p, q) > eps)
            res ^= 1;
        if (eq(cross(p, q), .0) and dot(p, q) < eps)
            return 1;
    }
    return (res ? 2 : 0);
}
```

string

kmp.hpp

md5: ae2331

```
// kmp[i] := max{ l ≤ i | s[:l] == s[(i+1)-l:i+1] }
// minimal period: i+1-kmp[i]
vector<int> KMP(string &s) {
    vector<int> a(SZ(s));
    rep(i, 1, n) {
        int g = a[i - 1];
        while (g and s[i] != s[g])
            g = a[g - 1];
        a[i] = g + (s[i] == s[g]);
    }
    return a;
}
```

manacher.hpp

md5: e14af2

```
vector<int> Manacher(const string &s) {
    string t;
    for (auto &c : s) {
        t.push_back(c);
        t.push_back('$');
    }
    t.pop_back();
    int i = 0, j = 0, n = t.size();
    vector<int> res(n);
    while (i < n) {
        while (i - j >= 0 and i + j < n and t[i - j] == t[i
+ j])
            j++;
        res[i] = j;
        int k = 1;
        while (i - k >= 0 and i + k < n and k + res[i - k] <
j) {
            res[i + k] = res[i - k];
            k++;
        }
        i += k;
        j -= k;
    }
    for (int i = 0; i < n; i++) {
        if (i & 1)
            res[i] = (res[i] / 2) * 2;
        else
            res[i] = ((res[i] + 1) / 2) * 2 - 1;
    }
    return res;
}
```

palindromictree.hpp

md5: cef5f5

```
struct PalindromicTree {
    struct Node {
        map<char, int> nxt;
        int link, len, id, cnt; // link: suffixである最長回文
の頂点
        Node() {}
        Node(int link, int len, int id, int cnt)
            : link(link), len(len), id(id), cnt(cnt) {}
    };
    vector<char> S;
    int cur;
    vector<Node> nodes;
    PalindromicTree() {
        nodes.push_back(Node(0, -1, -1, 0));
        nodes.push_back(Node(0, 0, -1, 0));
        cur = 0;
    }
    void push(char c) {
        S.push_back(c);
        int k = find(SZ(S) - 1, cur, S);
        if (nodes[k].nxt.count(c)) {
            cur = nodes[k].nxt[c];
            nodes[cur].cnt++;
            return;
        }
        nodes[k].nxt[c] = SZ(nodes);
        cur = SZ(nodes);
        nodes.push_back(
            Node(-1, nodes[k].len + 2, (SZ(S) - 1) -
nodes[k].len - 1, 1));
        if (nodes.back().len == 1)
            nodes.back().link = 1;
        else {
            int n = find(SZ(S) - 1, nodes[k].link, S);
            nodes.back().link = nodes[n].nxt[c];
        }
    }
    vector<array<int, 3>> get_freq() {
        vector<array<int, 3>> ret;
        for (int i = SZ(nodes) - 1; i >= 2; i--) {
            ret.push_back({nodes[i].len, nodes[i].id,
nodes[i].cnt});
            nodes[nodes[i].link].cnt += nodes[i].cnt;
        }
        return ret;
    }

  private:
    template <typename T> int find(int last, int k, const T
&S) const {
        for (;;) {
            int i = last - 1 - nodes[k].len;
            if (i >= 0 and S[i] == S[last])
                break;
            k = nodes[k].link;
        }
        return k;
    }
};
```

rollinghash.hpp

md5: 1ca953

```
struct RollingHash {
    using ull = unsigned long long;
    const ull MOD = 2305843009213693951LL;
    const ull base;
    vector<ull> hashed, power;
    static constexpr ull mask(ll a) { return (1ULL << a) -
1; }
    inline ull mul(ull a, ull b) const {
        __uint128_t ans = __uint128_t(a) * b;
        ans = (ans >> 61) + (ans & MOD);
        if (ans >= MOD)
            ans -= MOD;
        return ans;
    }
    static inline ull genbase() { return RNG(0,
2305843009213693950LL); }
    RollingHash() = default;
    RollingHash(const string &s, ull base) : base(base) {
        ll n = s.size();
        hashed.assign(n + 1, 0);
        power.assign(n + 1, 0);
        power[0] = 1;
        for (ll i = 0; i < n; i++) {
            power[i + 1] = mul(power[i], base);
            hashed[i + 1] = mul(hashed[i], base) + s[i];
            if (hashed[i + 1] >= MOD)
                hashed[i + 1] -= MOD;
        }
    }
    ull get(ll l, ll r) const {
        ull ret = hashed[r] + MOD - mul(hashed[l], power[r -
```

```
l]);
        if (ret >= MOD)
            ret -= MOD;
        return ret;
    }
    ll LCP(const RollingHash &b, ll l1, ll r1, ll l2, ll r2)
{
        ll len = min(r1 - l1, r2 - l2);
        ll low = -1, high = len + 1;
        while (high - low > 1) {
            ll mid = (low + high) / 2;
            if (get(l1, l1 + mid) == b.get(l2, l2 + mid))
                low = mid;
            else
                high = mid;
        }
        return low;
    }
};
```

run.hpp

md5: 22e8c7

```
struct Run {
    using P = pair<int, int>;
    using T = pair<int, P>;
    string s;
    vector<vector<P>> run;
    vector<T> sub(string t, string r) {
        int x = t.size(), y = r.size();
        string p = t, q = r;
        reverse(ALL(p));
        q.insert(q.end(), ALL(t));
        q.insert(q.end(), ALL(r));
        vector<int> v1 = Zalgo(p), v2 = Zalgo(q);
        vector<T> res;
        rep(i, 0, x) {
            int a = x - i, b = i - v1[a], c = max(0, y -
v2[y + i]);
            if (x + y - b - c >= 2 * a)
                res.push_back({a, {b, c}});
        }
        return res;
    }
    void rec(int lb, int rb) {
        if (rb - lb <= 1)
            return;
        int m = (lb + rb) / 2;
        string t(s.begin() + lb, s.begin() + m),
            r(s.begin() + m, s.begin() + rb);
        auto ret = sub(t, r);
        for (auto &[a, bc] : ret) {
            auto [b, c] = bc;
            run[a].push_back({lb + b, rb - c});
        }
        reverse(ALL(t));
        reverse(ALL(r));
        ret = sub(r, t);
        for (auto &[a, bc] : ret) {
            auto [b, c] = bc;
            run[a].push_back({lb + c, rb - b});
        }
        rec(lb, m);
        rec(m, rb);
    }
    Run(string _s) : s(_s) {
        int n = s.size();
        run.clear();
        run.resize(n + 1);
        rec(0, n);
        auto cmp = [&](P p, P q) {
            return P{p.first, -p.second} < P{q.first, -
q.second};
        };
        rep(t, 1, n + 1) {
            sort(ALL(run[t]), cmp);
            int mx = -1;
            vector<P> tmp;
            for (auto p : run[t])
                if (chmax(mx, p.second))
                    tmp.push_back(p);
            run[t] = tmp;
        }
    }
};
```

suffixarray.hpp

md5: 297590

```
vector<int> SA(string &s) {
    int n = SZ(s);
    vector<int> sa(n);
    iota(ALL(sa), 0);
    sort(ALL(sa),
         [&](int i, int j) { return s[i] == s[j] ? i > j :
s[i] < s[j]; });
    vector<int> rank(n), c(n), cnt(n);
    rep(i, 0, n) c[i] = s[i];
    for (int len = 1; len < n; len <<= 1) {
        rep(i, 0, n) {
            if (i == 0 or c[sa[i - 1]] != c[sa[i]]) {
                rank[sa[i]] = i;
            } else {
                if (sa[i - 1] + len >= n or
                    c[sa[i - 1] + len / 2] != c[sa[i] + len
/ 2]) {
                    rank[sa[i]] = i;
                } else {
                    rank[sa[i]] = rank[sa[i - 1]];
                }
            }
        }
        iota(ALL(cnt), 0);
        copy(ALL(sa), c.begin());
        rep(i, 0, n) {
            int j = c[i] - len;
            if (j >= 0)
                sa[cnt[rank[j]]++] = j;
        }
        swap(rank, c);
    }
    return sa;
}

vector<int> LCP(string &S,
                vector<int> &sa) { //
lcp[i]=LCP(s[sa[i]:],s[sa[i+1]:])
    int n = SZ(S);
    vector<int> rank(n), lcp(n - 1);
    rep(i, 0, n) rank[sa[i]] = i;
    int h = 0;
    rep(i, 0, n) {
        if (rank[i] + 1 < n) {
            int j = sa[rank[i] + 1];
            while (max(i, j) + h < n && S[i + h] == S[j +
h])
                h++;
            lcp[rank[i]] = h;
            if (h > 0)
                h--;
        }
    }
    return lcp;
}
```

zalgo.hpp

md5: 187b0d

```
template <typename T> vector<int> Zalgo(T &s) {
    int n = s.size();
    vector<int> res(n);
    for (int i = 1, j = 0; i < n; i++) {
        if (i + res[i - j] < j + res[j])
            res[i] = res[i - j];
        else {
            int k = max(0, j + res[j] - i);
            while (i + k < n && s[k] == s[i + k])
                k++;
            res[i] = k;
            j = i;
        }
    }
    res[0] = n;
    return res;
}
```

memo

kazoeage.md

数列一覧

- 第一種スターリング数 $c(n,k)$: $1,\ldots,n$ の順列で巡回置換 k 個に分割できるものの個数
  - $c(n,k)=c(n-1,k-1)+(n-1)c(n-1,k)$
  - $[x^k]x(x+1)\cdots(x+n-1)=c(n,k)$
  - $\sum_k 2^k c(n,k)=(n+1)!, \sum_k(-1)^k c(n,k)=0$
- 第二種スターリング数 $S(n,k)$: $1,\ldots,n$ を k 個の区別しない集合に分割する方法の数
  - $S(n,k)=S(n-1,k-1)+kS(n-1,k)$
  - $S(n,k)=\frac{1}{k!}\sum_{m=1}^{k}(-1)^{k-m}\binom{k}{m}m^n$
- ベル数 B_n : $1,\ldots,n$ をいくつかの集合に分割する方法の数
  - $\exp(\exp x-1)=\sum_n B_n\frac{x^n}{n!}$
  - $B_{n+1}=\sum_{k=0}^{n}\binom{n}{k}B_k$
- カタラン数 C_n : 長さ $2n$ の valid なカッコ列の個数
  - $C_n=\frac{1}{n}\binom{2n}{n}=[x^n]\frac{1-\sqrt{1-4x}}{2x}$
  - $C_{n+1}=\frac{2(2n+1)}{n+2}C_n=\sum_{k=0}^{n}C_kC_{n-k}$
- モンモール数 a_n : $1,\ldots,n$ の順列で $p_i\neq i$ となる個数
  - $a_n=(n-1)(a_{n-1}+a_{n-2}), a_n=na_{n-1}+(-1)^n$
- 分割数 P_n : n を正整数の和で表す方法の数
  - $\sum_n P_nx^n=\prod_{n\geq 1}(1-x^n)=\sum_{n=-\infty}^{\infty}(-1)^n x^{n(3n+1)/2}$

FPS 公式

$[x^n]\frac{1}{(1-x)^d}=\binom{n+d-1}{n}$

$[x^n]\frac{1}{\sqrt{1-4x}}=\binom{2n}{n}$

$\frac{1}{k}(\exp x-1)^k=\sum_n S(n,k)\frac{x^n}{n!}$ (第二種スターリング数)

$\frac{\exp(-x)}{1-x}=\sum_n a_n\frac{x^n}{n!}$ (モンモール数)

$[x^n]C(x)^k=\frac{k}{n+k}\binom{2n+k-1}{n}$

Lagrange 反転

$F(x),G(x)\in\mathbb{C}(x)$ s.t. $F(G(x))=G(F(x))=x$ について、
$[x^n]G(x)=\frac{1}{n}[x^{n-1}]\{\frac{x}{F(x)}\}^n$

LGV lemma

$\{a_1,\ldots,a_n\}$ を始点集合、$\{b_1,\ldots,b_n\}$ を終点集合とする DAG があり、「$i<j$ なら $a_i\to b_j$ の path と $a_j\to b_i$ の path は必ず交差する」と仮定する。このとき、互いに交わらない $a_i\to b_i$ の path の組の個数は、 $a_i\to b_j$ の path の個数を c_{ij} として $\det(c_{ij})$ に等しい。

matching.md

Hall の定理

頂点が U,V に分割された二部グラフ上で、「U をカバーするマッチングが存在」 $\iff$ 「$\forall A\subset U, |A|\leq|\delta(A)|$」

二部グラフのマッチング関連

	サイズ	構成
最小点被覆	M	$V\setminus M$ から L 到達不可能+R 到達可能
独立集合	$V-M$	上の補集合
最小辺被覆	孤立点ないなら $V-M$	M +入らない頂点で Greedy

Dilworth の定理

推移的な DAG 上で、最小 path 被覆のサイズと最大独立集合のサイズは同じ。
c.f. (DAG 上の最小 path 被覆のサイズ)= N -(頂点倍化した二部グラフの最大マッチング)が成り立つ。

optimization.md

燃やす埋める

変形前の制約	変形後の制約
x が 0 のとき z 失う	(x,T,z)
x が 0 のとき z 得る	無条件で z 得る; (S,x,z)
x が 1 のとき z 失う	(S,x,z)
x が 1 のとき z 得る	無条件で z 得る; (x,T,z)
x,y がともに 0 のとき z 得る	無条件で z 得る; $(S,w,z),(w,x,\infty),(w,y,\infty)$
x,y がともに 1 のとき z 得る	無条件で z 得る; $(w,T,z),(x,w,\infty),(y,w,\infty)$

LP 双対

$\max c^Tx$ s.t. $Ax=b, x\geq 0$ と $\min b^Ty$ s.t. $A^Ty\geq c$ の最適解は、存在すれば一致。

Erdös-Gallai Theorem

$\{d_i\}$ 単調減少列が n 頂点の木の次数列となるためには、「$\sum d_i$ が偶数かつ $\forall k,\sum_{i\leq k}d_i\leq k(k-1)+\sum_{i>k}\min(d_i,k)$」が必要十分。

Gale-Ryser Theorem

$\{a_i\},\{b_i\}$ 単調減少列が $n\times n$ の 01 matrix の各行各列の総和の列となるためには、「$\sum a_i=\sum b_i$ かつ $\forall k,\sum_{i\leq k}a_i\leq\sum\min(b_i,k)$」が必要十分。